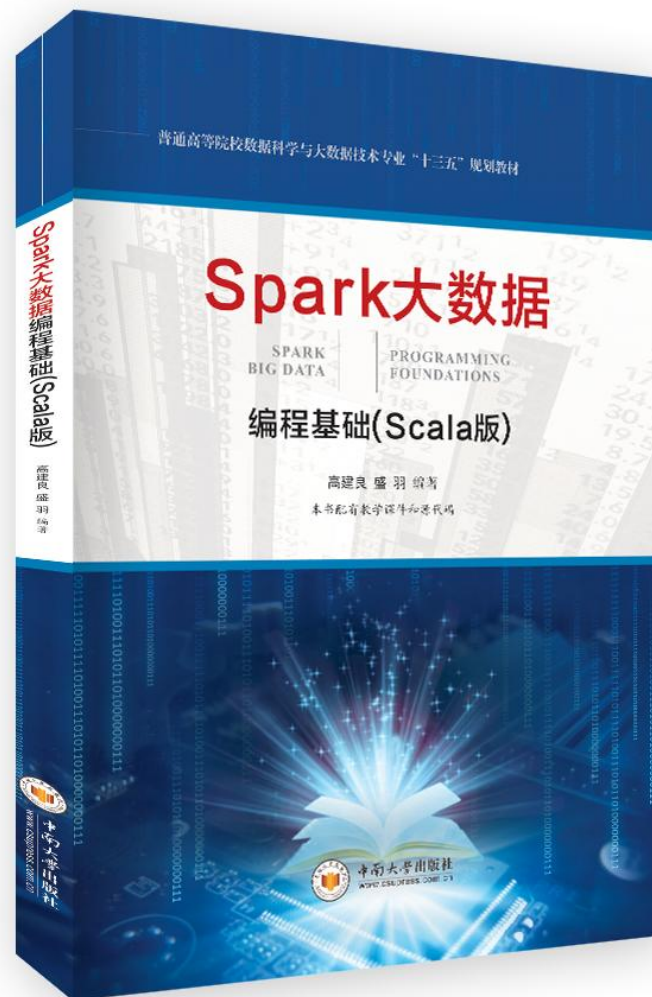
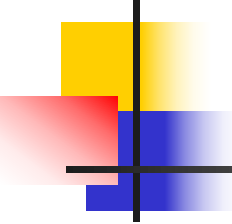


Spark大数据 编程基础 (Scala版)

<http://aibigdata.csu.edu.cn>

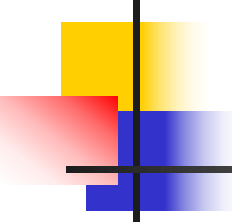




Spark 大数据编程基础（Scala版）

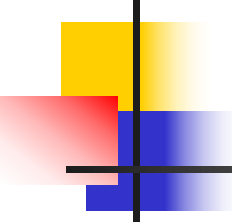
第八章 Spark GraphX

在大数据时代，大规模图无处不在。GraphX是Spark中的图计算组件，提供了丰富的图算法库，使得大规模图计算变得简洁高效。本章介绍GraphX的基本构成，主要包括底层的GraphX的图存储，中间层的GraphX图操作和上层的图算法库，本章对这些构成部分分别进行了介绍。



主要内容

- 8.1 GraphX简介
- 8.2 GraphX图存储
- 8.3 GraphX图操作
- 8.4 内置的图算法
- 8.5 GraphX实现经典图算法
- 8.6 GraphX实例分析
- 8.7 本章小结



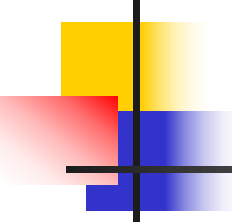
8.1 GraphX简介

图（**Graph**）结构可以抽象表示现实世界中的很多关系。例如，在社交网络中，图的“顶点”表示社交网络中的人，“边”表示人与人的关系。此外，地图导航、网页链接关系和消费者网上的购物等都可以用图抽象表示。基于图的数据结构衍生出了许多基础算法，如遍历、最小生成树、最短路径等。



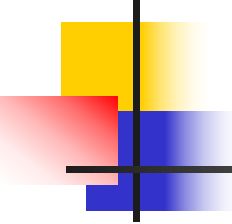
8.1 GraphX简介

同时这些数据结构也扩展出许多应用场景，比如淘宝的推荐商品、**Facebook**的推荐好友等。但是由于图结构中数据内部存在较高的关联性，在图计算时会引入大量连接和聚集等操作，严重消耗计算资源，对这些算法进行优化显得非常重要。



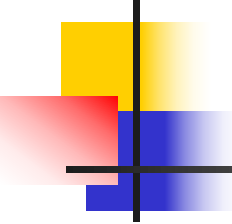
8.1 GraphX简介

为了提高图计算的速度，一些企业、社区都提供了并行化的图计算解决方案，常见的有GraphX、Pregel、PowerGraph、Graphlib等。其中，GraphX作为Spark的图计算组件，在实际应用中表现尤为突出。



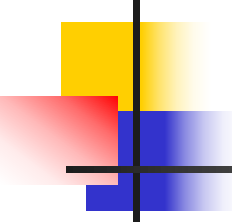
8.1 GraphX简介

GraphX是一个分布式图处理框架，基于**Spark**平台提供对图计算和图挖掘简洁易用而丰富多彩的接口，满足了大规模图处理的需求。与其他的图处理系统和图数据库相比，基于图概念和图处理原语的**GraphX**，优势在于既可以将底层数据看成一个完整的图，也可以对边**RDD**和顶点**RDD**使用数据并行处理原语。



8.1 GraphX简介

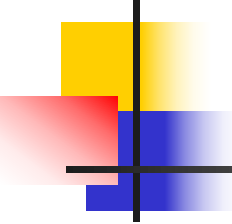
GraphX的核心抽象是Resilient Distributed Property Graph，一种点和边都带属性的有向多重图。它扩展了Spark RDD的抽象，一方面依赖于RDD的容错性实现了高效的健壮性，另一方面GraphX可以与Spark SQL、Spark ML等无缝地结合使用，例如使用Spark SQL收集的数据可以交由GraphX进行处理，而GraphX在计算时可以和Spark ML结合完成深度数据挖掘等操作，这些是其他图计算框架所没有的。



8.1 GraphX简介

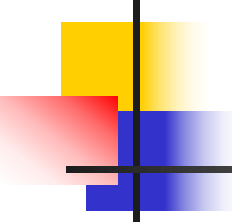
同时，Spark GraphX代码非常简洁，其实现架构大致分为三层，如图8-1所示。

- (1) **存储层**：该层定义了GraphX的数据模型，包括顶点RDD、边RDD和三元组Triplets；介绍了GraphX点分割技术、不同的分区策略以及数据存储方式；介绍了图计算过程使用的数据存储结构，如路由表、重复顶点视图等。



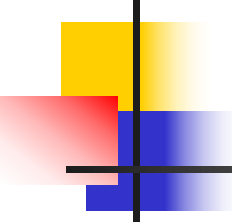
8.1 GraphX简介

(2) 操作层：在抽象类**Graph**及其实现类**GraphImpl**两个类中定义了构建图操作、转换操作、结构操作、聚合操作和缓存操作等；另外，在**GraphOps**类中也实现了图基本属性操作和连接操作等。**Graph**类是最重要的一个类，也是一个抽象类，**Graph**类中一些没有具体实现的内容是由**GraphImpl**完成，**GraphOps**是一个协同工作类。



8.1 GraphX简介

(3) 算法层: GraphX根据实现层和操作层实现了常用的算法, 如PageRank、统计三角形数、计算连通向量等, 同时, 提供了一个优化的Pregel API迭代算法, 大部分的GraphX内置算法是用Pregel实现的。



8.1 GraphX简介

GraphX的框架如图8-1所示，其中存储层为操作层和算法层提供了基础数据及其存储方式，操作层大大精简了图计算的程序设计，算法层实现了基础的图算法，为后续的图计算应用提供了有力的支持。

8.1 GraphX简介

算法

PageRank

SVDPlusPlus

TriangleCount

ConnectedComponents

...

Pregel

操作

GraphOps

Graph

GraphImpl

存储

VertexRDD

EdgeRDD

RDD[EdgeTriplet]

PartitionStrategy

Edge

EdgeTriplet

MessageToPartition

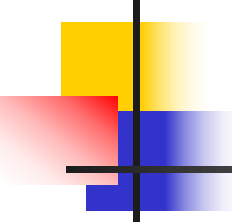
VertexPartition

EdgePartition

RoutingTablePartition

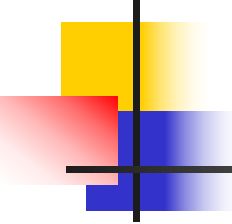
ReplicatedVertexView

图 8-1 GraphX实现架构图



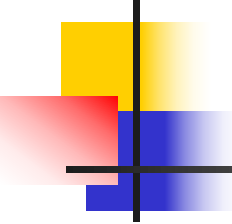
8.2 GraphX图存储

在Spark里抽象了一个通用的数据结构RDD（Resilient Distributed Dataset，弹性分布式数据集）来表示运算中需要的各种数据类型。本节将介绍GraphX的三种RDD数据模型，可以方便的表示图和存储图。GraphX采用点分割方式存储图，在集群上均匀分布边，进而更均匀地平衡整个群集的数据。



8.2.1 GraphX的RDD

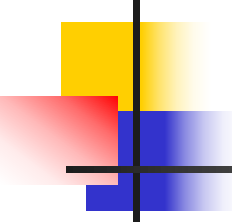
在GraphX中，图的基础类为Graph，其主要包含两个弹性分布式数据集（RDD）：一个为边RDD，另一个为点RDD。可以利用给定的边RDD和顶点RDD构建一个图。一旦建立好图，就可以用函数edges()和vertices()来访问边和顶点的集合。GraphX还特有一种数据结构是函数triplets()返回EdgeTriplet[VD,ED]类型的RDD。



8.2.1 GraphX的RDD

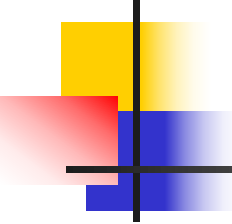
GraphX基本的数据结构为顶点(Vertex)、边(Edge)和三元组(Triplet)。

- 顶点(Vertex): 包含顶点ID和顶点数据(VD), 可以表示为(顶点ID, 顶点数据VD)
- 边(Edge): 包含源顶点和目标顶点ID以及边数据(ED), 可以表示(源顶点ID, 目标顶点ID, 边数据ED)。



8.2.1 GraphX的RDD

- 三元组(Triplets): 它是边的子类, 在边的基础上存储了边的源顶点和目标顶点的数据, 可以表示为(源顶点, 目标顶点, 边数据)。



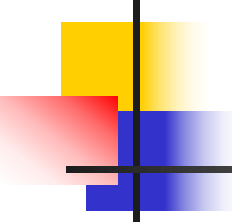
8.2.1 GraphX的RDD

(1) 顶点RDD (VertexRDD)

在GraphX中顶点数据抽象为VertexRDD，VertexRDD继承RDD[(Vertex, VD)]，RDD的类型是VertexId和VD，其中的VD是属性的类型。顶点分区定义如代码8-1所示：

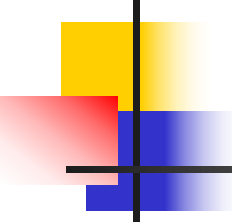
代码 8-1

```
class VertexPartition[@specialized ( Long, Int, Double)  VD : ClassTag](  
    val index: VertexIdToIndexMap,  
    val values: Array[VD],  
    val mask: BitSet,  
    private val activeSet: Option[VertexSet] = None )
```



8.2.1 GraphX的RDD

Index表示顶点**ID**与顶点数据在顶点数据数组中索引映射，**values**表示存储顶点数据的数组，**mask**边数过滤**index**中顶点的掩码，**activeSet**表示活跃顶点的**ID**。



8.2.1 GraphX的RDD

(2) 边RDD (EdgeRDD)

在GraphX中边数据抽象为EdgeRDD，该EdgeRDD[ED, VD]继承来自RDD[Edge[ED]]，以各种分区策略将边划分成不同的块。在每个分区中，边属性和邻接结构分别存储，这使得更改属性值时能够实现最大限度的复用。边分区的定义如代码8-2所示：



8.2.1 GraphX的RDD

代码 8-2 ↗

```
class EdgePartition[@specialized (Char, Int, Boolean, Byte, Long, Float, Double)  ED : ClassTag](  
    val srcIds: Array[VertexId], ↗  
    val dstIds: Array[VertexId], ↗  
    val data: Array[ED], ↗  
    val index: PrimitiveKeyOpenHashMap[VertexId,Int] ) ↗
```

srcIds存储所有源顶点的ID，dstIds存储所有目的顶点的ID，data存储所有边数据，index为顶点ID在源顶点ID数组中的起始索引。

8.2.1 GraphX的RDD

(3) Triplets (三元组)

Triplets的属性有：源顶点ID，源顶点属性、边属性、目标顶点ID、目标顶点属性，其可以看成是对**Vertices**和**Edges**做了**join**操作。如果需要用顶点、顶点属性及顶点关联边属性，则可创建**Triplets**的RDD。如图8-2所示。

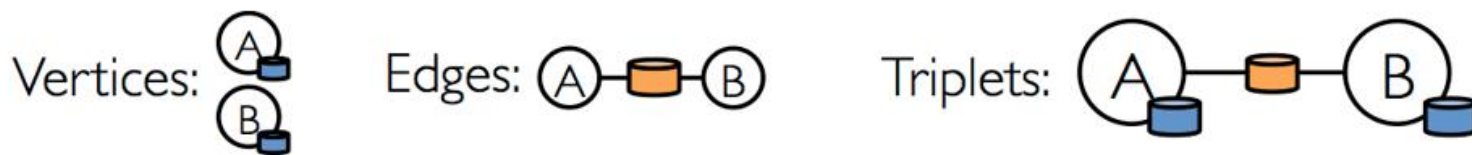
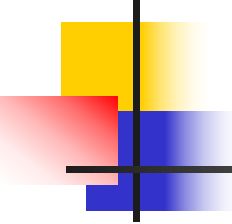
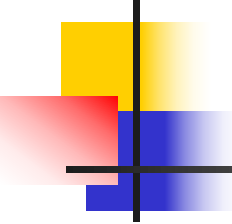


图8-2 Triplets对Vertices和Edge进行的连接操作



8.2.2 GraphX图分割

图分割方式一般有**边分割**和**点分割**两种分割方式，与边分割相比，点分割在性能上取得了重大提升，目前基本上被业界广泛接受并使用。



8.2.2 GraphX图分割

- **边分割（Edge Cut）**：每个顶点都存储一次，但是有的边会被打断分到不同机器上。这样做的好处是节省储存空间；缺点是对图计算进行基于边的计算时，对于一条两个顶点被分到不同机器上的边来说，要跨机器通信传输数据，内网通信流量大。如图8-3左图有3个分区，其分区1中包含顶点A和顶点C，分区2中包含顶点B，而分区3中包含顶点D，这些顶点在集群中只存储一次。

8.2.2 GraphX图分割

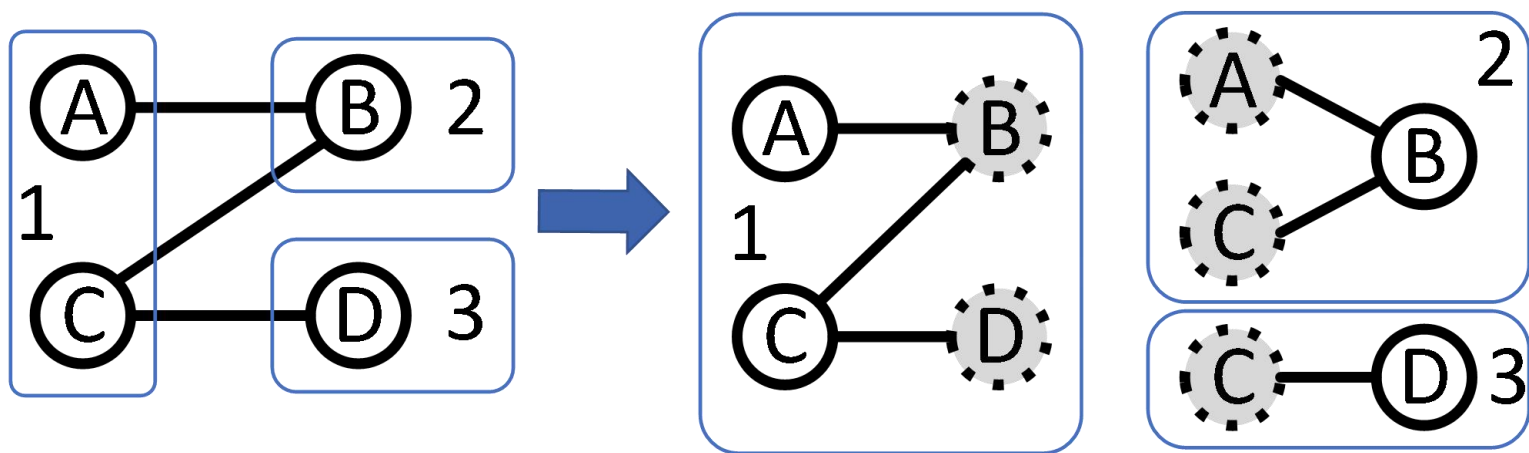
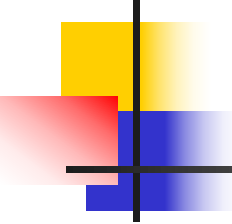


图8- 3 边分割示意图



8.2.2 GraphX图分割

- 点分割（**Vertex Cut**）：每条边只存储一次，都只会出现在一台机器上。邻居多的点会被复制到多台机器上，增加了存储开销，同时会引发数据同步问题。点分割的好处是可以大幅度减少内网通信量，如图8-4中右边图分为3个分区，其中分区1包含顶点A、B和边AB，分区2包含顶点B、C和边BC，分区3包含顶点C、D和边CD，这些边在集群中只存储一次，而顶点可能重复存储。

8.2.2 GraphX图分割

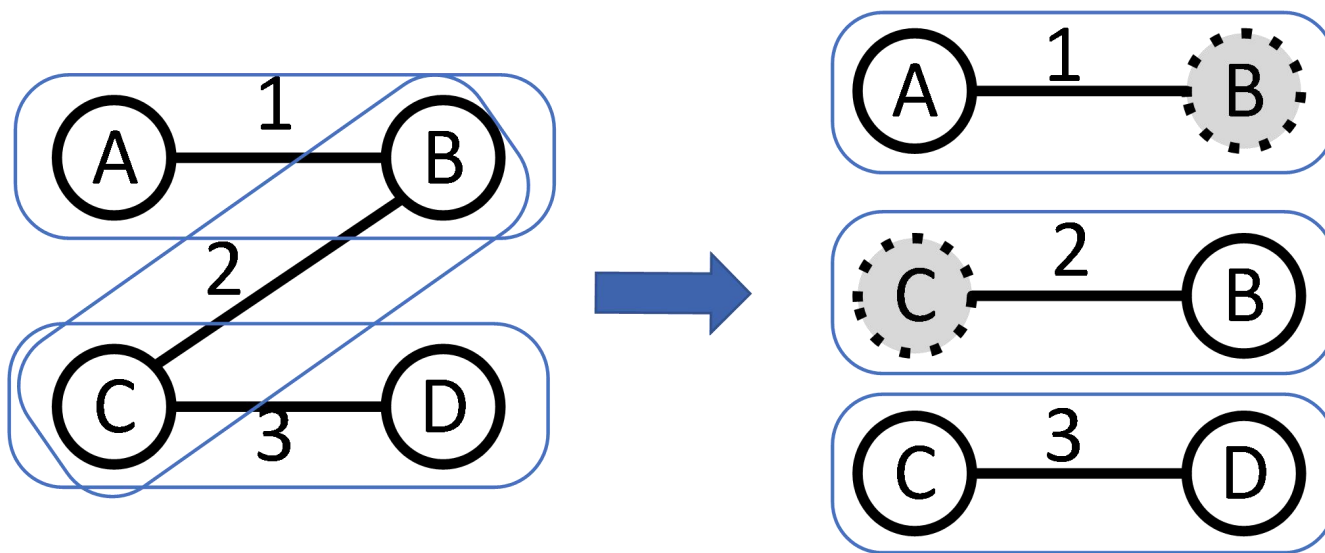
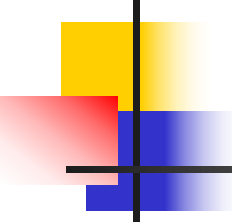


图8- 4 点分割示意图



8.2.2 GraphX图分割

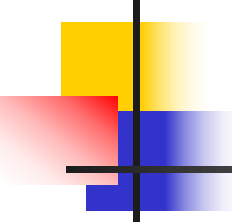
虽然两种方法互有利弊，但是现在点分割更占优势，各种分布式计算框架都将自己底层的存储形式变成了点分割。其主要有以下两个原因：

（1）磁盘价格下降，存储空间不再是问题，而内网的通信资源没有突破性进展，集群计算时内网带宽是宝贵的，时间比磁盘更珍贵。这点就类似于常见的空间换时间的策略。



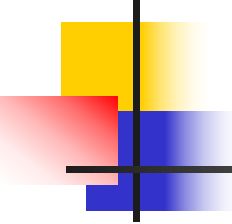
8.2.2 GraphX图分割

(2) 在当前的应用场景中，绝大多数网络是“无尺度网络”，遵循幂律分布，不同点的邻居数量相差非常悬殊。而边分割会使那些多邻居的点所相连的边大多数被分到不同的机器上，这样的数据分布会使得内网带宽更加捉襟见肘，于是边分割存储方式渐渐被抛弃了。



8.2.2 GraphX图分割

GraphX使用的Vertex Cut，即对顶点进行切分，在集群上均匀分布边，进而更均匀地平衡整个群集的数据。第一次构建图时，要么用`Graph.apply()`[与`Graph()`等价]，要么用`GraphLoader`，这都使用了默认的初始边分区，但图整体上并不以任何逻辑形式分区。这会导致性能很差，此外，像`triangleCount()`这样的一些函数要求图分区才能正确运算。



8.2.2 GraphX图分割

GraphX支持边上的4种不同的Partition（分区）策略：

- （1）RandomVertexCut：边随机分布；
- （2）CanonicalRandomVertexCut：要求两个顶点间如果有多条边则分在同一分区中；
- （3）EdgePatition1D：从一个顶点连出去的边在同一分区；
- （4）EdgePatition2D：边通过元组(srcId,dstId)划分为两维的坐标系统。

8.2.2 GraphX图分割

以图8-5为例对Partition进行说明：

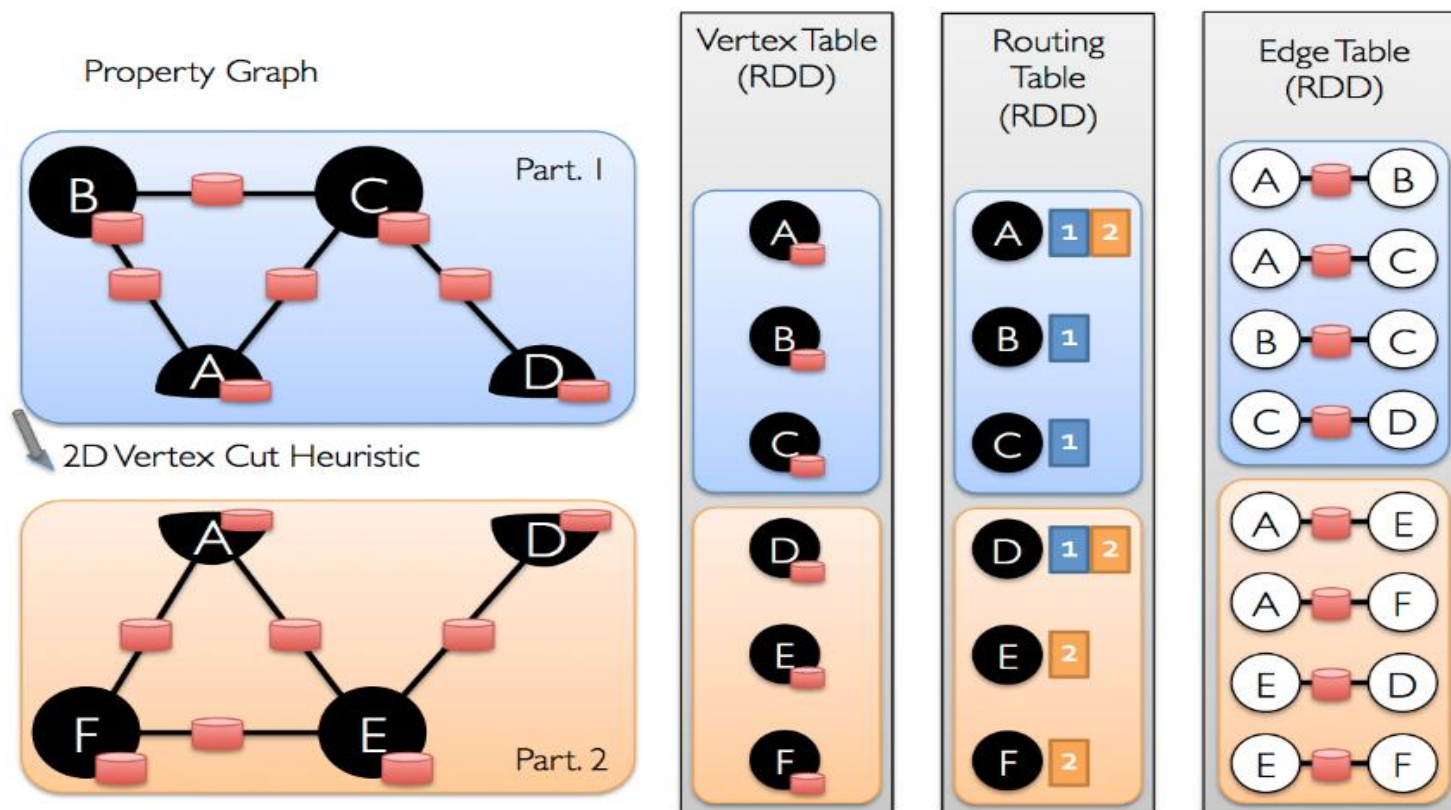
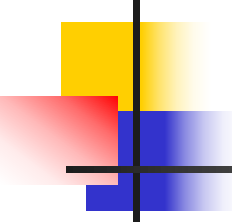
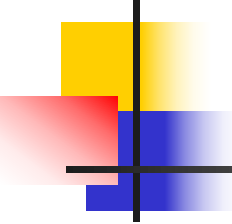


图 8-5 点分割方式储存图



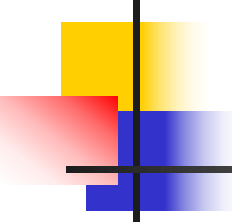
8.2.2 GraphX图分割

图8-5左侧的属性图被分割成两个部分，**Vertex Table**的信息表明其中A、B、C在一个分割区中，D、E、F在另一个分割区中；**Edge Table**表明每个分区中不同的边，除此之外，还有一个**Routing Table**部分，其记录了节点的路由信息，例如A在part1和part2中都出现，也就是说在做**mapVertices**和**mapEdges**等操作的时候内部结构不会发生改变。



8.3 GraphX图操作

GraphX常用的操作分为几类：构建图操作、基本属性操作、连接操作、转换操作、结构操作、聚合操作、缓存操作等同时提供了基于谷歌Pregel API的迭代算法。基本上的图操作，只需要一个函数调用即可完成，精简了图计算的程序设计。



8.3 GraphX图操作

更详细全面的GraphX的API请参照官方文档。链接为：

<http://spark.apache.org/docs/latest/api/scala/index.html#org.apache.spark.package>

<http://spark.apache.org/docs/latest/graphx-programming-guide.html#summary-list-of-operators>

8.3.1 构建图操作

GraphX常用的构建图操作如表8-1所示。

表 8-1 构建图操作

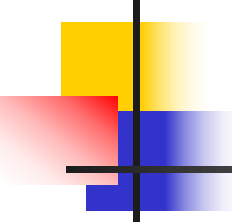
接口	描述
<pre>fromEdgeTuples[VD:ClassTag](rawEdges:RDD[(VertexId,VertexId)], defaultValue:VD, uniqueEdges:Option[PartitionStrategy]=None, edgeStorageLevel:StorageLevel= StorageLevel.MEMORY_ONLY, vertexStorageLevel:StorageLevel= StorageLevel.MEMORY_ONLY):Graph[VD,Int]</pre>	通过一组边构建图，其中边由源顶点和目的顶点组成，在参数中提供顶点和边的存储级别，默认情况下均存储在内存中 · <code>defaultValue</code> 为顶点的默认数据，用于当顶点在边 RDD 存在但是顶点 RDD 中不存在的时候，为顶点提供默认值 · <code>uniqueEdges</code> 参数用于提供一个分区策略，对生成的图结构进行分区操作，若提供了该参数，图中重复的边会被合并，重复边的属性相加得到合并后的属性，若不提供该参数，图中重复的边不做处理 · <code>edgeStorageLevel</code> 参数为边的存储级别，默认情况为存储在内存中 · <code>vertexStorageLevel</code> 参数为顶点的存储级别，默认情况为存储在内存中



8.3.1 构建图操作

续表 8-1 构建图操作

<pre>fromEdges[VD:ClassTag,ED:ClassTag](edges:RDD[Edge[ED]], defaultValue:VD, edgeStorageLevel:StorageLevel= StorageLevel.MEMORY_ONLY, vertexStorageLevel:StorageLevel= StorageLevel.MEMORY_ONLY):Graph[VD,ED]</pre>	通过一组边构建图，其中边的数据包含了顶点数据和边的值，该接口提供了顶点的默认值，在参数中提供了顶点和边的存储级别，默认情况下均为存储在内存中 · <code>edgeStorageLevel</code> 参数为边的存储级别，默认情况为存储在内存中 · <code>vertexStorageLevel</code> 参数为顶点的存储级别，默认情况为存储在内存中
<pre>apply[VD:ClassTag,ED:ClassTag](vertices:RDD[(VertexId,VD)], edges:RDD[Edge[ED]], defaultVertexAttr:VD=null.asInstanceOf[VD], edgeStorageLevel:StorageLevel= StorageLevel.MEMORY_ONLY, vertexStorageLevel:StorageLevel= StorageLevel.MEMORY_ONLY):Graph[VD,ED]</pre>	该方法是 <code>Graph</code> 在创建图使用的默认方法，在参数中提供了顶点和边的存储级别，默认情况下均存储在内存中 · <code>RDD[(VertexId,VD)]</code> 顶点 RDD 包含顶点的 ID 和数据 · <code>RDD[Edge[ED]]</code> 边 RDD 包含边的源顶点和目标顶点 ID 以及边的属性



8.3.1 构建图操作

其中，最常用的是**apply**操作，即根据创建的顶点RDD以及边RDD，构建一个属性图**Graph[VD,ED]**，以图8-6为例，实现过程如代码8-3所示，介绍图的构建过程。后续的图操作都是基于此代码的**myGraph**属性图来演示。

8.3.1 构建图操作

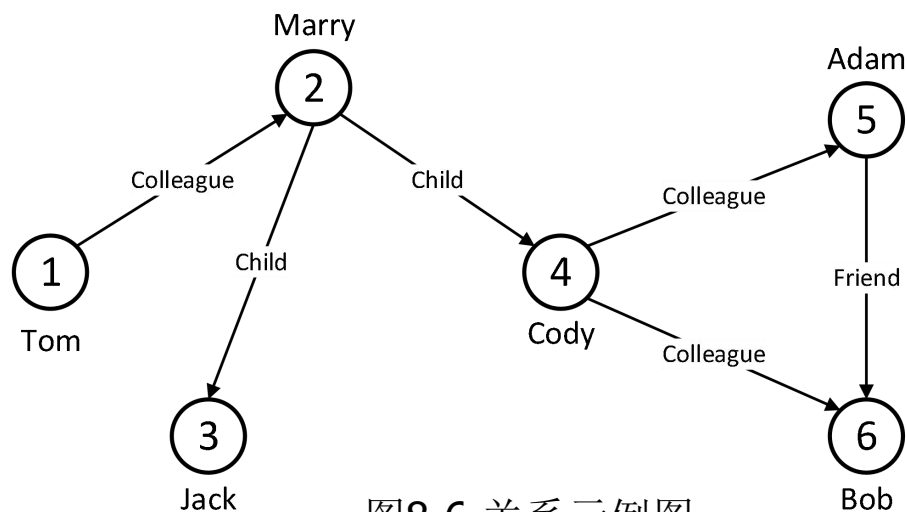


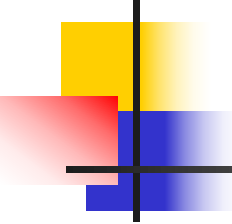
图8-6 关系示例图

表8-2 Vertex Table

Id	Property(V)
1	Tom
2	Marry
3	Jack
4	Cody
5	Adam
6	Bob

表8-3 Edge Table

SrcId	DstId	Property(E)
1	2	Colleague
2	3	Child
2	4	Child
4	5	Colleague
4	6	Colleague
5	6	Friend



8.3.1 构建图操作

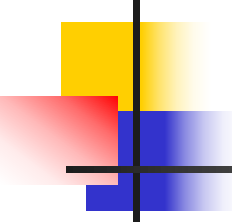
在spark-shell中实现构建图的操作如代码8-3所示：

代码 8-3

```
scala> import org.apache.spark.graphx._ //首先将GraphX导入
scala> val myVertices =
sc.parallelize(Array((1L,"Tom"),(2L,"Marry"),(3L,"Jack"),(4L,"Cody"),(5L,"Adam
"),(6L,"Bob"))) //构造VertexRDD

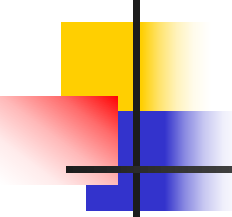
scala> val myEdges =
sc.parallelize(Array(Edge(1L,2L,"Colleague"),Edge(2L,3L,"Child"),Edge(2L,4L,"Ch
ild"),Edge(4L,5L,"Colleague"),Edge(4L,6L,"Colleague"),Edge(5L,6L,"Friend"))) //
构造EdgeRDD

scala> val myGraph=Graph(myVertices,myEdges) //构造图Graph[VD,ED]
```

8.3.1 构建图操作

注意，当一个Scala类或者对象中定义了函数`apply()`，在调用`apply()`时可以省略`apply`，即`Graph.apply()`简写成`Graph()`，所以`Graph()`看起来像一个构造函数，但实际上它是在调用`apply()`函数。



8.3.2 基本属性操作

GraphX的基本属性操作如表8-4所示。

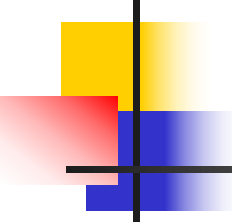
表8-4 基本属性操作

接口	描述
vertices:VertexRDD[VD]	整个图的顶点 RDD
edges:Edge[ED,VD]	整个图的边 RDD
triplets:RDD[EdgeTriplet[VD,ED]]	整个图的三元组
numVertices:Long	整个图顶点总数
numEdges:Long	整个图边总数
inDegrees:VertexRDD[Int]	整个图所有顶点的入度，若顶点无入度，则不出现在结果中
outDegrees:VertexRDD[Int]	整个图所有顶点的出度，若顶点无出度，则不出现在结果中
degreesRDD(edgeDirection:EdgeDirection): VertexRDD[Int]	计算相邻顶点的度，edgeDirection 参数控制收集方向
collectNeighborIds(edgeDirection: EdgeDirection):VertexRDD[Array[VertexId]]	收集每个顶点的相邻顶点的 ID 数据，edgeDirection 用于控制收集的方向

8.3.2 基本属性操作

续表8-4 基本属性操作

<code>collectNeighbors(edgeDirection: EdgeDirection):VertexRDD[Array[(VertexId,VD)]]</code>	收集每个顶点的相邻顶点的数据，当图中顶点的出入度较大时，可能会占用很大的存储空间，参数 <code>edgeDirection</code> 用于控制收集的方向
<code>collectEdges(edgeDirection:EdgeDirection): VertexRDD[Array[Edge[ED]]]</code>	收集每个顶点边的数据，参数 <code>edgeDirection</code> 用于控制收集的方向
<code>JoinVertices[U:ClassTag](table:RDD[(VertexId,U)]) (mapFunc:(VertexId,VD,U)=>VD):Graph[VD,ED]</code>	使用输入的顶点数据更新新生成的顶点数据。将当前图的数据和输入的顶点数据做内连接操作，过滤输入数据中不存在的顶点，并对过滤的结果数据使用自定义函数计算，如果输入数据中没有包含图中某些顶点数据，则在新图中使用原图的顶点数据
<code>filter[VD2:ClassTag,ED2:ClassTag](preprocess:Graph[VD,ED]=>Graph[VD2,ED2],epred:(EdgeTriplet[VD2,ED2]) => Boolean=(x:EdgeTriplet[VD2,ED2])=>true,vpred:(VertexId,VD2)=>true):Graph[VD,ED]</code>	根据条件对图进行过滤操作，首先使用预处理函数(<code>preprocess</code>)对图进行转换操作生成新的顶点和边的数据，然后新的图数据上使用 <code>epred</code> 和 <code>vpred</code> 函数分别对边和顶点进行过滤操作，最后返回过滤后的结果数据
<code>pickRandomVertex():VertexId</code>	在图中随机获取一个顶点并返回该顶点的ID



8.3.2 基本属性操作

根据代码8-3定义的myGraph属性图进行以下基本属性操作：

1.从构建图中查看顶点集合：

代码8-4

```
scala> myGraph.vertices.collect  
res0: Array[(org.apache.spark.graphx.VertexId, String)] =  
Array((1,Tom), (2,Marry), (3,Jack), (4,Cody), (5,Adam), (6,Bob))
```



8.3.2 基本属性操作

2. 从构建图中查看边集合:

代码**8-5**

```
scala> myGraph.edges.collect
```

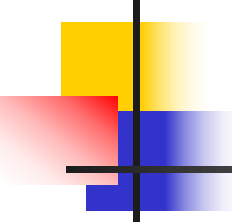
```
res1: Array[org.apache.spark.graphx.Edge[String]] = Array(Edge(1,2,Colleague),  
Edge(2,3,Child), Edge(2,4,Child), Edge(4,5,Colleague), Edge(4,6,Colleague),  
Edge(5,6,Friend))
```

3. 从构建的图中查看triplet形式数据:

代码**8-6**

```
scala> myGraph.triplets.collect
```

```
res2: Array[org.apache.spark.graphx.EdgeTriplet[String,String]] =  
Array(((1,Tom),(2,Marry),Colleague), ((2,Marry),(3,Jack),Child),  
((2,Marry),(4,Cody),Child), ((4,Cody),(5,Adam),Colleague),  
((4,Cody),(6,Bob),Colleague), ((5,Adam),(6,Bob),Friend))
```

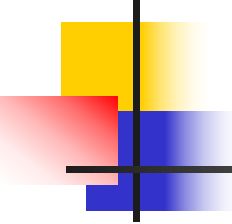


8.3.2 基本属性操作

函数triplets()返回EdgeTriplet[VD,ED]类型的RDD，包含边的源顶点和目标顶点的引用，可以很方便地对边和顶点的属性进行访问。EdgeTriplet类提供了访问边（以及边属性数据）以及源顶点和目标顶点属性数据的方法，如表8-5所示：

表8-5 EdgeTriplet的关键字段

字段	描述
attr	边属性
srcId	边的源顶点 ID
srcAttr	边的源顶点属性数据
dstId	边的目标顶点 ID
dstAttr	边的目标顶点属性数据



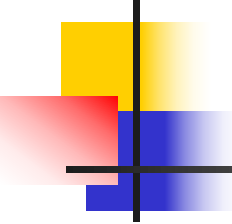
8.3.2 基本属性操作

4.Degrees操作

GraphX中，把Degree分成三种：inDegrees、outDegrees、degrees，分别表示计算图中的入度、出度和度数（入度和出度的和）。查看图入度的实例如代码8-7所示。

代码8-7

```
scala> myGraph.inDegrees.collect  
res3: Array[(org.apache.spark.graphx.VertexId, Int)] = Array((2,1), (3,1), (4,1),  
(5,1), (6,2))
```



8.3.2 基本属性操作

代码8-7结果表明，若某个顶点没有入度，那么结果中不会出现该顶点。以代码8-8为例，找出所有节点中度数最大的节点，并显示出来。

代码8-8

```
//定义max函数找出两两节点中度数较大的节点
scala> def max(a :(VertexId,Int), b :(VertexId,Int)): (VertexId,Int)={
    if(a._2 > b._2) a else b
}
max: (a: (org.apache.spark.graphx.VertexId, Int), b: (org.apache.spark.graphx.VertexId, Int))(org.apache.spark.graphx.VertexId, Int)

//使用reduce操作对两两元素进行归约操作，找出所有节点中度数最大的节点
scala> myGraph.degrees.reduce(max)
res4: (org.apache.spark.graphx.VertexId, Int) = (4,3)
```


8.3.2 基本属性操作

根据图8-6可得，度为3的顶点ID为2和4，当最大顶点度数的顶点有多个时，随机输出一个，所以结果显示顶点4的度最大为3。

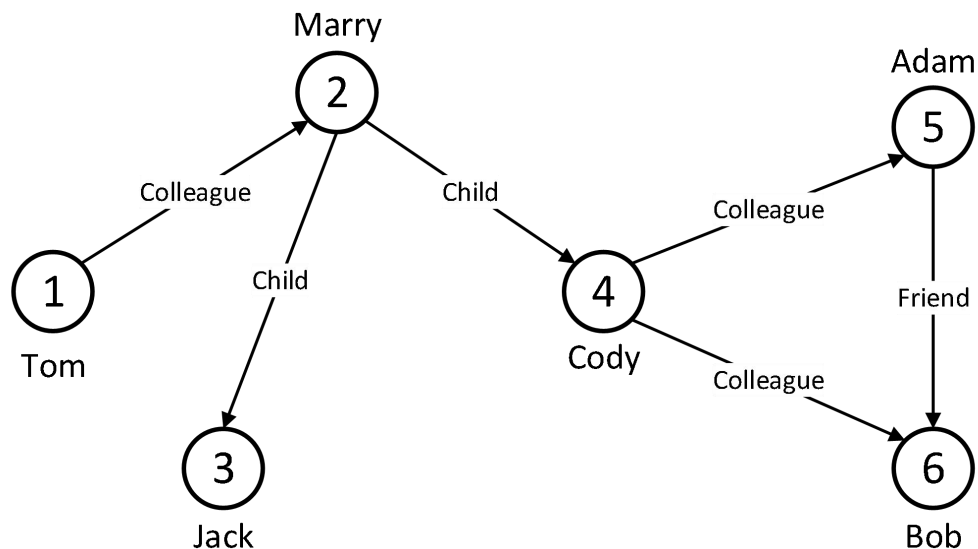
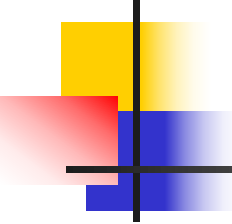
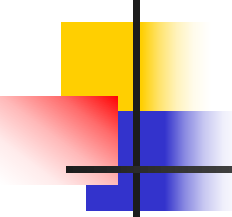


图8-6 关系示例图



8.3.3 连接操作

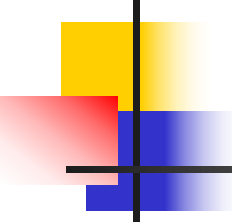
许多情况下，需要将图与外部获取的**RDD**进行连接。比如将一个额外的属性添加到一个已经存在的图上，或者将顶点属性从一个图导出到另一图中。**GraphX**的连接操作如表8-6所示。



8.3.3 连接操作

表8-6 连接操作

接口	描述
<code>outerJoinVertices[U:ClassTag, VD2:ClassTag](other:RDD[(VertexId,U)])(mapFunc:(VertexId,VD,Option[U])=>VD2)(implicit eq:VD == VD2 = null):Graph[VD2,ED]</code>	通过该接口可以实现当前图和其他图进行连接操作，并在连接结果上使用 mapFunc 函数进行计算，形成一个新图。该方法为 GraphX 核心操作接口
<code>joinVertices[U](table: RDD[(VertexId, U)])(mapFunc: (VertexId, VD, U) => VD): Graph[VD, ED]</code>	此运算符连接输入 RDD 的顶点，并返回一个新图，新图的顶点属性通过用户自定义的 map 功能作用在被连接的顶点上。没有匹配的 RDD 保留原始值。



8.3.3 连接操作

在基本属性操作的代码**8-7**可知，若顶点没有相应的度信息，则不会出现在结果中，在代码**8-9**中使用了 **outerJoinVertices** 操作，将没有度信息的顶点属性变为0。



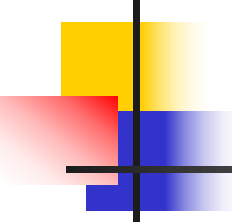
8.3.3 连接操作

代码8-9

```
scala> val outDegrees: VertexRDD[Int] = myGraph.outDegrees
outDegrees: org.apache.spark.graphx.VertexRDD[Int] = VertexRDDImpl[25] at RDD at
VertexRDD.scala:57

scala> val degreeGraph = myGraph.outerJoinVertices(outDegrees) { (id, oldAttr,
outDegOpt) => outDegOpt match {
    case Some(outDeg) => outDeg
    case None => 0 //没有度信息则为0
  }
}
degreeGraph: org.apache.spark.graphx.Graph[Int,String] =
org.apache.spark.graphx.impl.GraphImpl@11544ddd

scala> degreeGraph.vertices.collect
res5: Array[(org.apache.spark.graphx.VertexId, Int)] = Array((4,2), (1,1), (6,0), (3,0), (5,1),
(2,2))
```

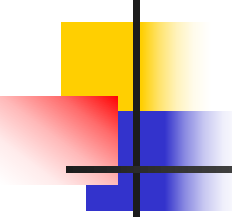


8.3.4 转换操作

和RDD的map操作类似，属性图的转换操作如表8-7所示。

表8-7 转换操作

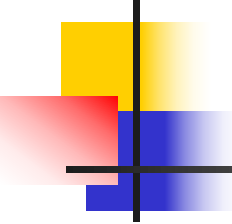
接口	描述
<code>partitionBy(partitionStrategy:PartitionStrategy):Graph[VD,ED]</code>	根据分区策略对图的边进行分区操作
<code>partitionBy(partitionStrategy:PartitionStrategy, numPartitions:Int):Graph[VD,ED]</code>	根据分区策略对图的边进行分区操作,和上一个操作相似
<code>mapVertices[VD2:ClassTag](map:(VertexId,VD)=>VD2):Graph[VD2,ED]</code>	对图中每个顶点的数据 VD 进行转换,生成新的顶点数据 VD2,从而生成一个新图,新图与原图具有相同结构
<code>mapEdges[ED2:ClassTag](map:Edge[ED]=>ED2):Graph[VD,ED2]</code>	和 mapVertices 相似, mapEdges 是针对图中的每一条边数据 ED 进行转换,从而生成一个新图,新图与原图具有相同结构



8.3.4 转换操作

续表8-7 转换操作

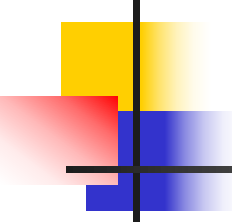
<pre>mapEdges[ED2:ClassTag](map: (PartitionID,Iterator[Edge[ED]])=> Iterator[ED2]):Graph[VD,ED2]</pre>	与 mapEdges 功能一致，不同之处在于 mapEdges 接口的方法每次只能处理一个边的数据，该方法可以处理一个分区所有边数据。在该方法中传入的是该分区编号 partitionID 和针对该分区的 Iterator，返回的是包含新的边值（ED）的分区的 Iterator，并且新分区中的数据与原始分区中的数据是一一对应的
<pre>mapTriplets[ED2:ClassTag](map: EdgeTriplets[VD,ED]=>ED2):Graph[VD,ED2]</pre>	使用该接口方法能够进行每条边的数据(ED)及边所连接的两个顶点数据(VD)进行计算，同时生成新的数据，不会改变图结构或图的值
<pre>mapTriplets[ED2:ClassTag](map:EdgeTriplets[VD,ED]=>ED2, tripletFields:TripletFields):Graph[VD,ED2]</pre>	与 mapTriplets 功能上一致，不同之处在于增加了参数 TripletFields，用于过滤不需要进行转换的 Triplets 对象，有助于提高处理效率



8.3.4 转换操作

续表8-7 转换操作

<pre>mapTriplets[ED2:ClassTag](map:(PartitionID,Iterator[EdgeTriplets[VD,ED]]) =>Iterator[ED2],tripletsFields:TripletFields):Graph[VD,ED2]</pre>	与 <code>mapTriplets</code> 功能上一致，不同之处在于 <code>mapTriplets</code> 接口方法，每次只能处理一个 <code>EdgeTriplet</code> 对象，而该方法可以进行批量处理一个分区所有 <code>EdgeTriplets</code> 对象。在该方法中传入的是该分区编号 <code>PartitionID</code> 和针对分区 <code>EdgeTriplet</code> 对象的 <code>Iterator</code>，返回的是包含新的 <code>Iterator[ED2]</code>和 <code>TripletField</code>，并且新分区中的数据与原始分区的数据是一一对应的
---	---

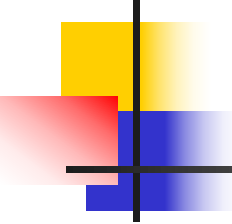


8.3.4 转换操作

以mapTriplets命令为例，对图8-6中满足条件的边进行增加属性，条件为：（1）属性中包含“Colleague”，（2）源顶点属性中包含字母“o”，然后以triplets形式输出。命令和结果如代码8-10所示：

代码8-10

```
scala> myGraph.mapTriplets(t => (t.attr, t.attr=="Colleague" &&
t.srcAttr.toLowerCase.contains("o"))).triplets.collect
res6: Array[org.apache.spark.graphx.EdgeTriplet[String,(String, Boolean)]] =
Array(((1,Tom),(2,Marry),(Colleague,true)), ((2,Marry),(3,Jack),(Child,false)),
((2,Marry),(4,Cody),(Child,false)), ((4,Cody),(5,Adam),(Colleague,true)),
((4,Cody),(6,Bob),(Colleague,true)), ((5,Adam),(6,Bob),(Friend,false)))
```

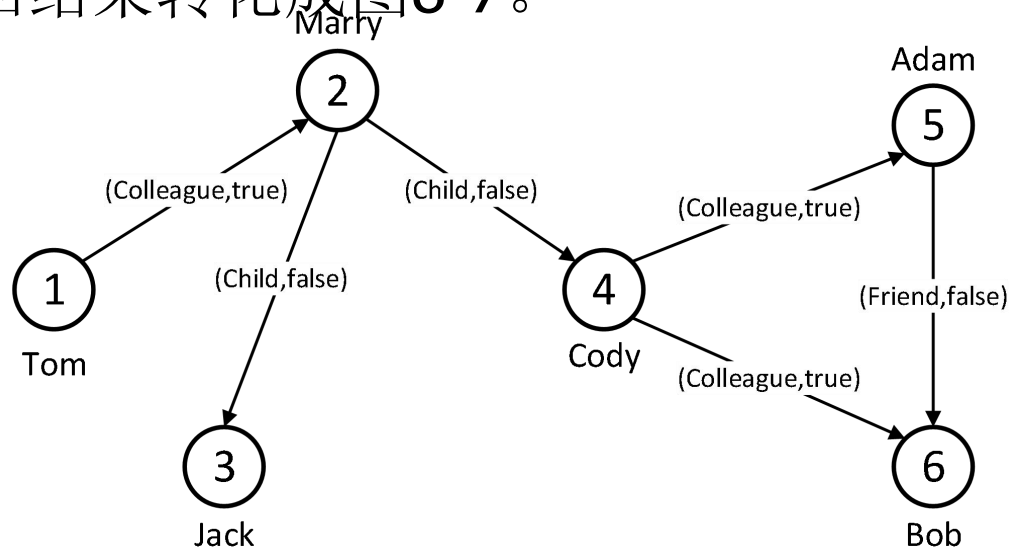


8.3.4 转换操作

`mapTriplets`有两个可选参数，这里只用了第一个参数，这个参数是一个匿名函数，传入一个`EdgeTriplet`对象作为输入参数，返回一个包含二元组（`String, Boolean`）的`Edge`类型。

8.3.4 转换操作

原始图myGraph（每条边不存在额外的布尔类型属性）依然存在，因为调用triplets()函数后生成的图对象并没有赋值给val或者var对象。但为了直观地看到转换，将代码8-10的返回结果转化成图8-7。



8.3.5 结构操作

GraphX的结构操作如表8-8所示。

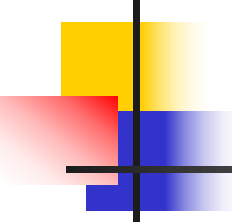
表8-8 结构操作

接口	描述
<code>reverse:Graph[VD,ED]</code>	将图中所有边进行反转，即若某条边连接两个顶点 a 和 b ，初始时边的方向为由 a 指向 b ，经过反转操作后边的方向由 b 指向 a
<code>subgraph(epred:EdgeTriplet[VD,ED]=>Boolean= (x=>true), vpred:(VertexId,VD)=>Boolean=((v,d)=>true)):Graph[VD,ED]</code>	在图中对顶点和边数据按照要求进行过滤生成一个新图。首先先生成图的边三元组，然后根据对图中源顶点、目的顶点和边的判定条件过滤边三元组，从而生成新的边数据，顶点数据可以通过对原图的顶点过滤得到，最后使用新的边数据和顶点数据生成新图
<code>mask[VD2:ClassTag,ED2:ClassTag](other:Graph[VD2,ED2]):Graph[VD,ED]</code>	在当前图中获取其他图中同样存在的顶点和边，获取的新图并保持顶点和边的数据与原图一致
<code>groupEdges(merge:(ED,ED)=>ED): Graph[VD,ED]</code>	将两个顶点之间多条边合并成一条，在合并过程中先使用 <code>partitionBy</code> 方法进行分区操作，以保证获取正确的结果



8.3.5 结构操作

subgraph操作将顶点和边的预测作为参数，并返回一个图，以subgraph操作为例，选择满足边属性为“Colleague”，实例如代码8-11所示，最终返回的图如图8-8所示。



8.3.5 结构操作

代码8-11

```
scala> val subGraph = myGraph.subgraph(each => each.attr == "Colleague")
```

```
subGraph: org.apache.spark.graphx.Graph[String,String] =  
org.apache.spark.graphx.impl.GraphImpl@48cbb4c5
```

```
scala> subGraph.vertices.collect
```

```
res7: Array[(org.apache.spark.graphx.VertexId, String)] = Array((4,Cody), (1,Tom),  
(6,Bob), (3,Jack), (5,Adam), (2,Marry))
```

```
scala> subGraph.edges.collect
```

```
res8: Array[org.apache.spark.graphx.Edge[String]] = Array(Edge(1,2,Colleague),  
Edge(4,5,Colleague), Edge(4,6,Colleague))
```

8.3.5 结构操作

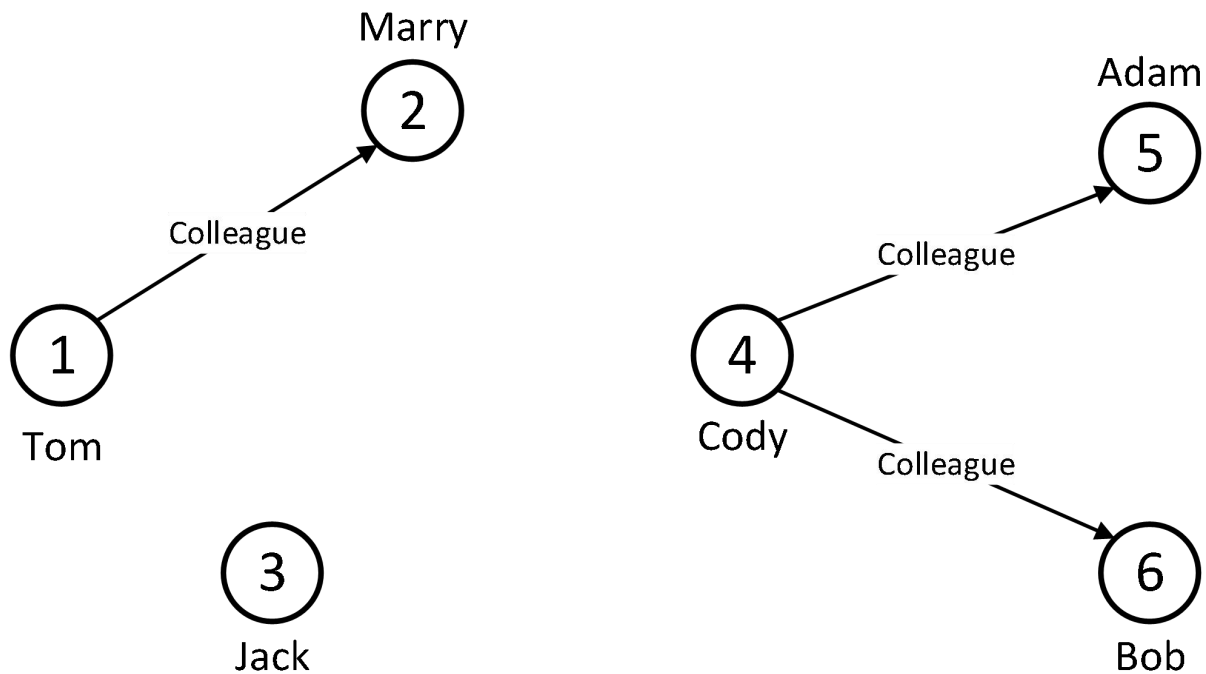
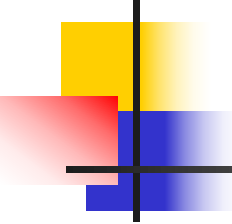


图8-8 满足操作的子图

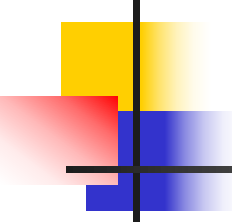


8.3.6 聚合操作

GraphX的聚合操作如表8-9所示。

表8-9 聚合操作

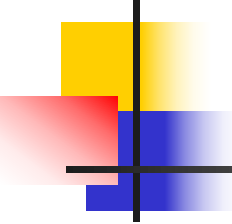
接口	描述
<code>aggregateMessages[A:ClassTag](sendMsg:EdgeContext[VD,ED,A]=>Unit,mergeMsg:(A,A)=>A,tripletFields:TripletField=TripletFields.All):VertexRDD[A]</code>	该方法是 GraphX 最重要的图操作方法之一，主要用来高效解决相邻边或相邻顶点之间的通信问题，例如：将与顶点相邻的边或顶点的数据聚集在顶点上，将顶点数据散发在相邻边上，它能够简单、高效地解决 PageRank 等图迭代应用。 该方法计算分为三步：①由边三元组生成消息；②向边三元组的顶点发送消息；③顶聚合收到的消息。
<code>aggregateMessagesWithActiveSet[A:ClassTag](sendMsg:EdgeContext[VD,ED,A]=>Unit,mergeMsg:(A,A)=>A,tripletFields:TripletFields,activeSetOpt:Option[(VertexRDD[_],EdgeDirection)]):VertexRDD[A]</code>	相对前一个方法 <code>aggregateMessages</code> 方法增加了 <code>activeSetOpt</code> 过滤参数，功能和计算过程类似



8.3.6 聚合操作

本小节主要介绍aggregateMessages操作。

aggregateMessages()方法是Spark GraphX中核心的聚合方法。为了理解在处理和聚集邻居顶点消息过程中的核心概念，这里仅考虑一个简单例子，即统计每个顶点的出度。选择间接处理每条边以及与边相关联的源顶点和目标顶点，让每条边发出消息到关联的源顶点，最终获得每条边的出度。

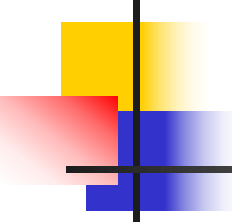


8.3.6 聚合操作

首先了解**aggregateMessages()**函数，它主要是用于迭代算法，基于邻边和顶点发送过来的消息不断更新每个顶点的状态，该函数有两个参数**sendMsg**和**mergeMsg**，分别提供了转换和聚合的能力。代码**8-12**给出了函数的定义。

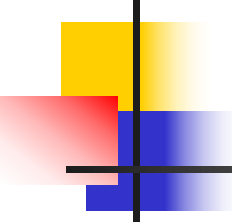
代码**8-12**

```
def aggregateMessages[Msg: ClassTag](  
    sendMsg: EdgeContext[VD, ED, Msg] => Unit,  
    mergeMsg: (Msg, Msg) => Msg,  
    tripletFields: TripletFields = TripletFields.All)  
    : VertexRDD[Msg]
```



8.3.6 聚合操作

注意这个函数的参数化类型：**Msg**。**Msg**表示函数返回结果数据的类型，本例中要对顶点传出的边数进行统计，**Msg**的具体类型选择**Int**类型。



8.3.6 聚合操作

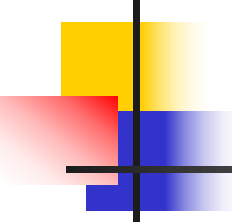
`sendMsg`函数以`EdgeContext`作为输入参数，无返回值，除此之外，其主要提供了两个消息的发送函数`sendToSrc`和`sendToDst`，分别表示将`Msg`类型的消息发送给源顶点和目标顶点。可以将`sendMsg`函数看成是map-reduce中的`map`函数。因为要统计顶点的出度，所以需计算每个顶点发出的边数，应将边上包含整数1的消息发送到源顶点。



8.3.6 聚合操作

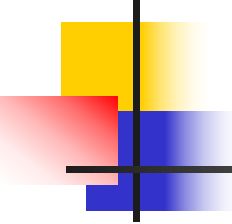
`mergeMsg`函数可以把每个顶点收到的所有消息都汇集起来，形成一条合并的消息。这个函数定义了如何将顶点收到的所有消息转化为最终需要的结果。因此可将`mergeMsg`函数看成是map-reduce中的`reduce`函数。

`aggregateMessages()`方法返回一个`VertexRDD[Msg]`对象。`VertexRDD`是一个包含二元组的RDD，包括了顶点的ID以及该顶点的`mergeMsg`操作的结果。没有收到消息的顶点不包含在返回的`VertexRDD`中。



8.3.6 聚合操作

此外，`aggregateMessages`采用了一个可选的 `tripletsFields`，该参数表示在 `EdgeContext` 所接受的数据，例如，只接受源顶点属性，而不接受目标顶点属性，那么只需使用 `TripletFields.Src`。该参数的默认值为 `TripletFields.All`，表示用户定义的 `sendMsg` 函数可以访问 `EdgeContext` 中的任何属性。



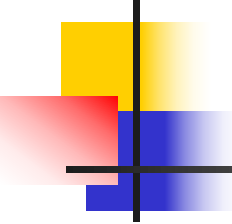
8.3.6 聚合操作

具体实例如代码8-13所示。

代码8-13

```
scala> myGraph.aggregateMessages[Int](_.sendToSrc(1), _+_).collect  
res9: Array[(org.apache.spark.graphx.VertexId, Int)] = Array((1,1), (2,2),  
(4,2), (5,1))
```

如果顶点不含有任何出边，则接收不到消息，因此不会出现在结果中。



8.3.7 缓存操作

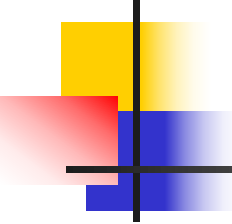
在Spark中，RDD默认并不保存在内存中。为了避免重复计算，当需要多次使用时，建议使用缓存。在迭代计算时，为了获得最佳性能，也可能需要清空缓存。默认情况下缓存的RDD和图表将保留在内存中，直到按照LRU（Least Recently Used）顺序被删除。对于迭代计算，之前迭代的中间结果将填补缓存。虽然缓存最终将被删除，但是内存中不必要的数据还是会使垃圾回收机制变慢。有效策略是，一旦缓存不再需要，应用程序立即清空中间结果的缓存。

8.3.7 缓存操作

GraphX的缓存操作如表8-10所示。

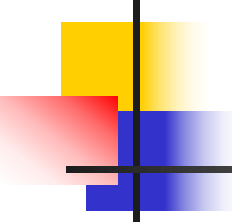
表8-10 缓存操作

接口	描述
<code>persist(newLevel:StorageLevel=StorageLevel.MEMORY_ONLY):Graph[VD,ED]</code>	使用指定的存储级别存储顶点和边的数据，忽略在此之前数据所指定的存储级别
<code>cache():Graph[VD,ED]</code>	将图缓存到内存中。由于在图的计算过程中，RDD 并不是一直都保存在内存中，然而在计算过程中，可能会多次用到图数据，为了避免开销，将图缓存到内存中
<code>unpersist(blocking:Boolean=true):Graph[VD,ED]</code>	释放存储中缓存的顶点数据，该方法多用于迭代生成新图之前对旧图数据的清理
<code>unpersistVertices(blocking:Boolean=true):Graph[VD,ED]</code>	释放内存中缓存的顶点数据，适用于只修改点的属性值，但会重复使用边进行计算的迭代操作。此方法可以释放先前迭代的顶点属性（当其不再需要的时候），提高 GC 性能
<code>checkpoint():Unit</code>	对图计算过程中的结果进行检查点操作，这些结果会暂时保存在可靠的存储中



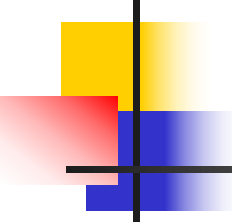
8.3.8 Pregel API

Spark GraphX中提供了方便开发者的基于谷歌Pregel API的迭代算法，因此可以用Pregel的计算框架来处理Spark上的图数据。GraphX的Pregel API提供了一个简明的函数式算法设计，用它可以在图中方便的迭代计算，如最短路径、关键路径、n度关系等，也可以通过对一些内部数据集的缓存和释放缓存操作来提升性能。



8.3.8 Pregel API

Pregel运算执行一系列的超步（**superstep**），每一个超步就是一轮单独的迭代。在每个超步内部，每个顶点的计算都是并行的，每个顶点会接收到它的邻居们在上一轮超步发送的消息的总和，然后计算顶点属性的新值；此外，在超步迭代的最后一步，每个顶点也会给它的邻居们发送消息。



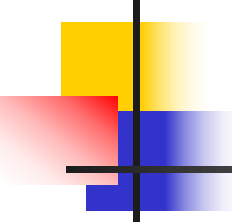
8.3.8 Pregel API

顶点也可以选择 not 发送消息；如果目标顶点没有从它的源顶点收到任何消息，它就不会参与下一个超步的运算。当没有消息发送时或是当前迭代次数大于默认迭代次数时，Pregel 运算符终止迭代并返回一个新的图。



8.3.8 Pregel API

图8-9展示了Pregel API完成一个超步的内部细节。上一个超步发送过来的消息会被顶点聚集在一起，然后由“消息合并”(`mergeMsg`)函数进行处理，这样每个顶点就只处理一条合并的消息（除非没有消息发送给这个顶点）。



8.3.8 Pregel API

`mergeMsg`函数返回结果消息但不会直接对顶点进行更新，它会把返回的结果消息作为参数传递给顶点处理程序`vprog`，`vprog`以顶点和消息作为输入，返回新的顶点数据以便被框架更新到顶点中。最后，每个顶点会沿着出边方向发送消息，当然也可以选择不发送，如果目标顶点没有从源顶点收到任何消息，它就不会参与下一个超步的运算。

8.3.8 Pregel API

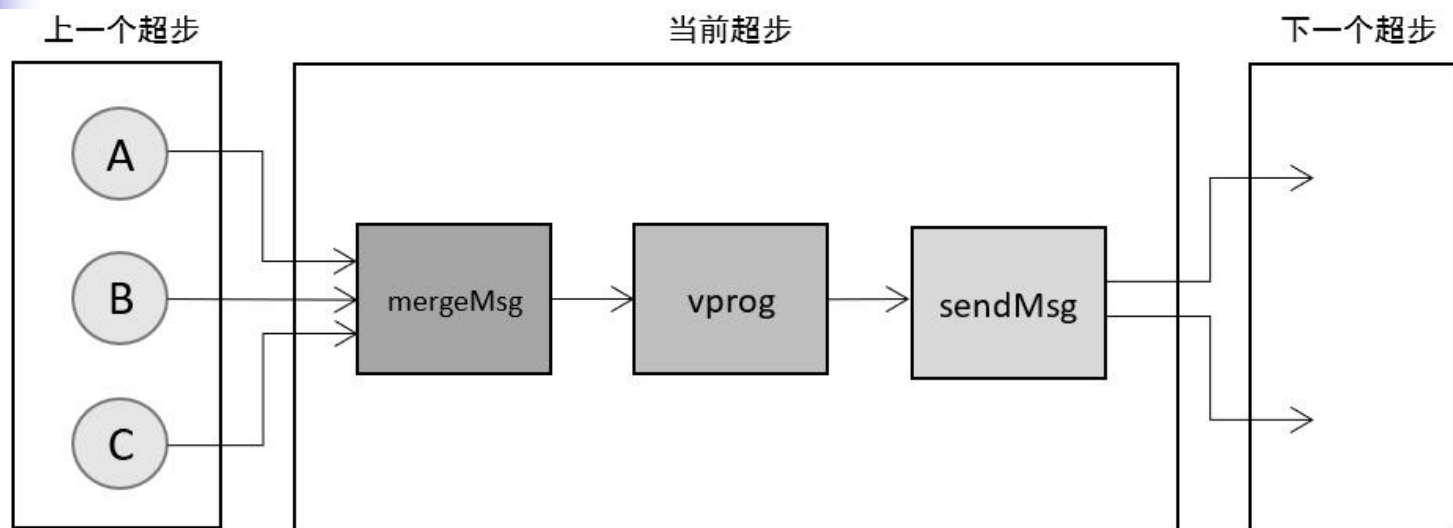


图 8-9 超步内部细节图

- A、B、C: 图中的顶点。
- mergeMsg: 上一个超步传递过来的顶点消息, 会通过自定义的mergeMsg函数聚合成单一的消息。

8.3.8 Pregel API

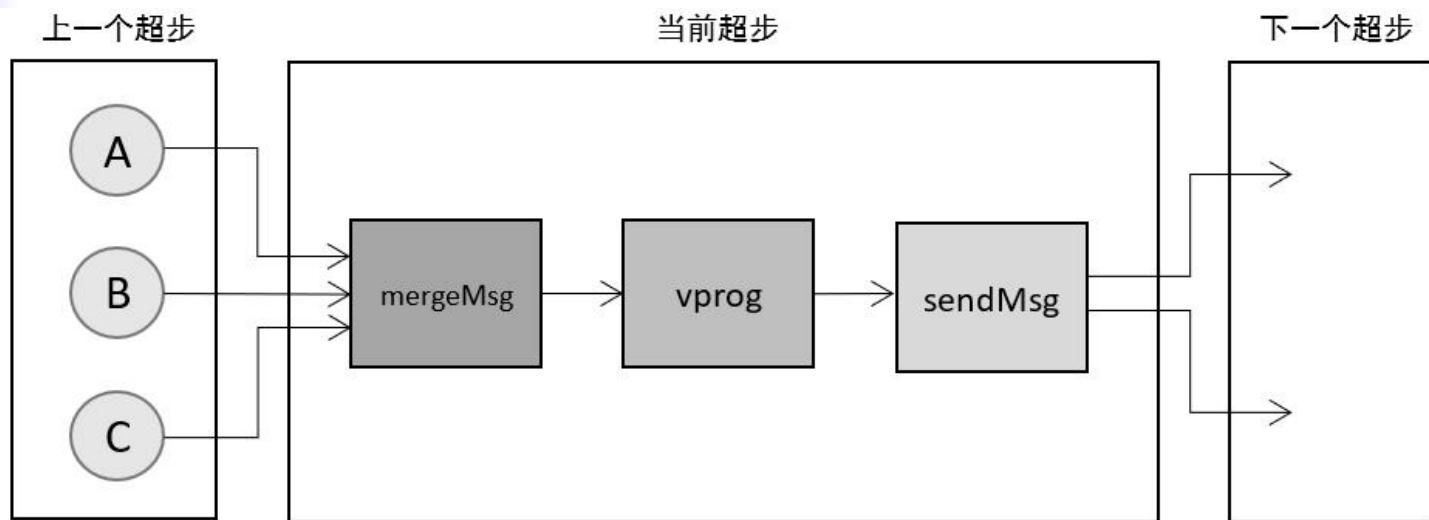
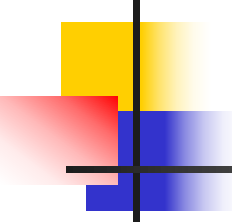


图 8-9 超步内部细节图

- **vprog**: 自定义的**vprog**函数决定如何用函数**mergeMsg**传来的消息来更新顶点数据。
- **sendMsg**: 自定义的**sendMsg**函数决定在下一个超步中哪些顶点会接受消息。

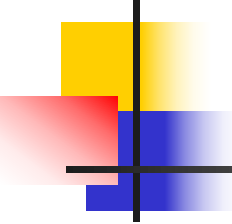


8.3.8 Pregel API

现在对Pregel的运行过程有大体的了解，代码8-14展示了pregel函数的定义。

代码8-14

```
def pregel[A]  
  (initialMsg: A,  
   maxIter: Int = Int.MaxValue,  
   activeDir: EdgeDirection = EdgeDirection.Out)  
  (vprog: (VertexId, VD, A) => VD,  
   sendMsg: EdgeTriplet[VD, ED] => Iterator[(VertexId, A)],  
   mergeMsg: (A, A) => A)  
  : Graph[VD, ED]
```



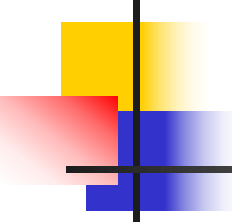
8.3.8 Pregel API

核心部分是三个函数：

(1) 节点处理消息的函数 $\text{vprog}: (\text{VertexId}, \text{VD}, A)$

$\Rightarrow \text{VD}$

用户自定义的函数，运行于每个节点上，和输入消息进行计算，生成新的顶点值，在第一次迭代时， vprog 在每个顶点上都执行一次，和默认输入消息进行计算，在之后的迭代时， vprog 只会在接收到消息的顶点上执行。



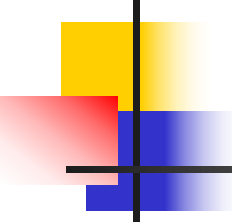
8.3.8 Pregel API

(2) 节点发送消息的函数 `sendMsg: EdgeTriplet[VD, ED] => Iterator[(VertexId, A)]`

用户自定义的函数，运行于每个活跃的边三元组上，产生发送给下一次迭代的消息。

(3) 消息合并函数 `mergeMsg: (A, A) => A`

用户自定义的函数，用于将两条发送给顶点的消息合并为一条消息。



8.3.8 Pregel API

(2) 节点发送消息的函数 `sendMsg: EdgeTriplet[VD, ED] => Iterator[(VertexId, A)]`

用户自定义的函数，运行于每个活跃的边三元组上，产生发送给下一次迭代的消息。

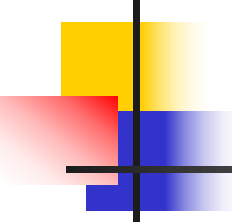
(3) 消息合并函数 `mergeMsg: (A, A) => A`

用户自定义的函数，用于将两条发送给顶点的消息合并为一条消息。



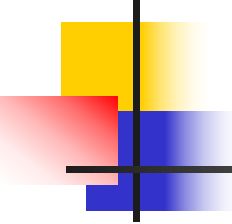
8.3.8 Pregel API

第一个参数列表中的参数是完成一些配置工作，
`initialMsg`传给顶点初始化值，一般设置为0值，`maxIter`
定义迭代次数，`activeDir`定义了过滤条件（终止条件），
可以分成`EdgeDirection.Out`，`EdgeDirection.In`，
`EdgeDirection.Either`，`EdgeDirection.Both`。



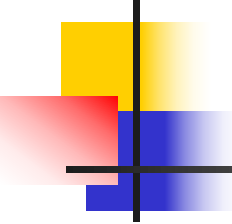
8.3.8 Pregel API

- **EdgeDirection.Out** — 当srcId收到来自上一轮迭代的消息时，就会调用**sendMsg**，这意味着把这条边当做srcId的“出边”。
- **EdgeDirection.In** — 当dstId收到来自上一轮迭代的消息时，就会调用**sendMsg**，这意味着把这条边当做dstId的“入边”。



8.3.8 Pregel API

- `EdgeDirection.Either` — 只要srcId或dstId收到来自上一轮迭代的消息时，就会调用sendMsg。
- `EdgeDirection.Both` — 只有srcId和dstId都收到来自上一轮迭代的消息时，才会调用sendMsg。



8.3.8 Pregel API

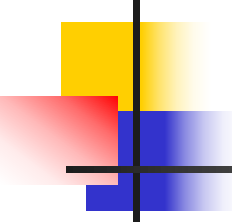
可以利用Pregel找出距离根节点最远的顶点和距离值。
在图8-6中，设定根节点为1(Tom)，具体实现如代码8-15所示。

代码8-15

```
scala> val g=Pregel(myGraph.mapVertices((vid,vd) =>
0),0,activeDirection=EdgeDirection.Out)((id:VertexId, vd:Int, a:Int) =>
math.max(vd,a),(et:EdgeTriplet[Int,String]) =>
Iterator((et.dstId,et.srcAttr+1)),(a:Int,b:Int) => math.max(a,b))

scala> g.vertices.collect
res10: Array[(org.apache.spark.graphx.VertexId, Int)] = Array((1,0), (2,1), (3,2),
(4,2), (5,3), (6,4))
```

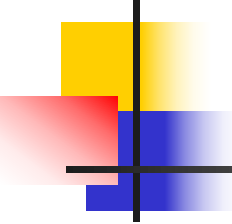
结果表明，Id为6的顶点与根节点的距离最远，距离值为4。



8.3.8 Pregel API

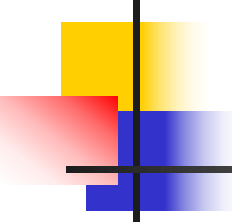
Pregel和aggregateMessages()函数都是GraphX图处理的基础，二者有着细微的差别。

- mergeMsg消息聚合函数的区别：在aggregateMessages中，mergeMsg函数将返回的消息结果直接对顶点更新，而pregel的mergeMsg函数将返回的消息结果传递给顶点处理程序vprog。



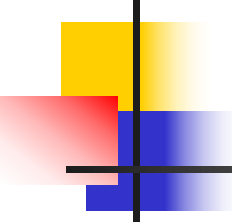
8.3.8 Pregel API

- `aggregateMessages`只需两个函数`sendMsg`和`mergeMsg`，而Pregel有了额外的`vprog`顶点处理程序，它可以更灵活地定义处理逻辑。有时候，传递的消息和顶点数据二者类型不一致，这时候就需要`vprog`。



8.3.8 Pregel API

- `sendMsg`函数的定义：`aggregateMessages`中是 `EdgeContext[VD, ED, Msg] => Unit`，而`pregel`中是 `EdgeTriplet[VD, ED] => Iterator[(VertexId, A)]`，`EdgeTriplet`包含了边及其两个顶点的消息，`EdgeContext`还额外包含了两个方法，`sendToSrc`和`sendToDst`。
- `aggregateMessages`返回的是一个`VertexRDD`对象，而`pregel`直接返回一个新的`Graph`对象。



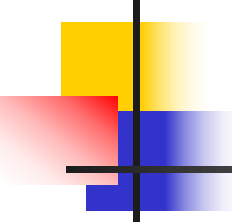
8.3.8 Pregel API

- `sendMsg`函数的定义：`aggregateMessages`中是 `EdgeContext[VD, ED, Msg] => Unit`，而`pregel`中是 `EdgeTriplet[VD, ED] => Iterator[(VertexId, A)]`，`EdgeTriplet`包含了边及其两个顶点的消息，`EdgeContext`还额外包含了两个方法，`sendToSrc`和`sendToDst`。
- `aggregateMessages`返回的是一个`VertexRDD`对象，而`pregel`直接返回一个新的`Graph`对象。



8.3.8 Pregel API

- Pregel的终止条件是不再有需要发送的消息，所以要求有更灵活的终止条件的算法可以用`aggregateMessages()`实现。

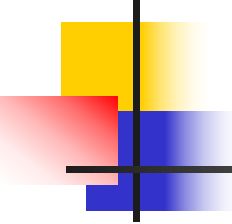


8.3.8 Pregel API

对于用pregel方法找出距离根顶点最远的顶点和距离值，用aggregateMessages()函数也可以实现，代码8-16展示迭代的aggregateMessages()，用于寻找与根顶点距离最远的顶点。

代码8-16

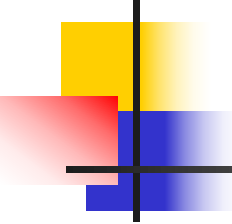
```
//定义sendMsg函数
scala> def sendMsg(ec:EdgeContext[Int,String,Int]):Unit = {
    ec.sendToDst(ec.srcAttr+1)
}
//定义mergeMsg函数
scala> def mergeMsg(a:Int,b:Int):Int = {
    math.max(a,b)
}
```



8.3.8 Pregel API

续代码8-16

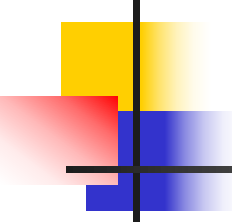
```
scala> def propagateEdgeCount(g:Graph[Int,String]):Graph[Int,String] = {  
    //生成新的顶点集  
    val verts = g.aggregateMessages[Int](sendMsg,mergeMsg)  
    val g2 = Graph(verts,g.edges)//根据新顶点集生成一个新图  
    //将新图g2和原图g连接，查看顶点的距离值是否有变化  
    val check = g2.vertices.join(g.vertices).  
    map(x=>x._2._1-x._2._2).  
    reduce(_+_)  
    //判断距离变化，如果有变化，则继续递归，否则返回新的图对象  
    if(check>0)  
        propagateEdgeCount(g2)  
    else  
        g  
}
```



8.3.8 Pregel API

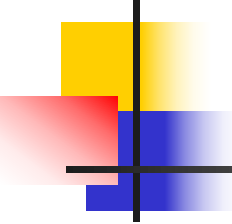
续代码**8-16**

```
//初始化距离值，将每个顶点的值设置为0  
scala> val newGraph = myGraph.mapVertices((_,_)=>0)  
scala> propagateEdgeCount(newGraph).vertices.collect  
res11: Array[(org.apache.spark.graphx.VertexId, Int)] = Array((1,0), (2,1), (3,2),  
(4,2), (5,3), (6,4))
```

8.4 内置的图算法

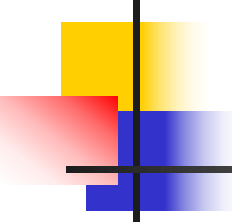
本节主要介绍这些基础算法，包括PageRank、三角形数、计算连通分量和标签传播算法。同时，也把图算法和机器学习结合起来，介绍唯一一个完全属于GraphX模块的机器学习算法SVDPlusPlus的用法。



8.4.1 PageRank

Google的创始人拉里·佩奇和谢尔盖·布林于1998年在斯坦福大学发明了PageRank算法。PageRank是一种根据网页之间相互超链接计算的技术，Google用它来体现网页的相关性和重要性，在搜索引擎优化操作中经常用来评估网页优化的成效因素之一。

PageRank算法最初是用于搜索引擎页面排名，其主要作用就是找到图中最重要的节点。



8.4.1 PageRank

其主要作用就是找到图中最重要的节点。主要应用包括：

- 基于人到人的关系图中通过权值排名区分出关键人物
- 基于“分享”的社交网络图中对其影响力做等级划分等

虽然PageRank算法的定义是一个递归调用，但是可以直接通过迭代计算，退出迭代的条件是：当前迭代是算法计算结果最稳定且为最终状态。

8.4.1 PageRank

算法流程

1) 用 $1/N$ 的页面排名值（PR）初始化每个顶点， N 为图中顶点总数。

2) 循环：

①每个顶点，沿着出边发送PR值 $1/M$ ， M 为当前顶点的出度。

②当每个顶点从相邻顶点收到其发送的PR值后，合理计算这些PR值作为当前顶点的新PR值。

③图中顶点的PR值与上一个迭代相比没有显著的变化，则退出迭代。

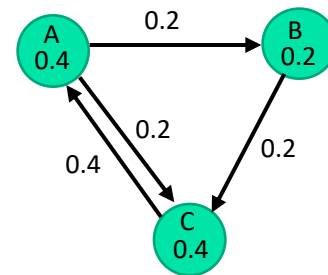


图 8-10 PageRank迭代后
稳定状态图



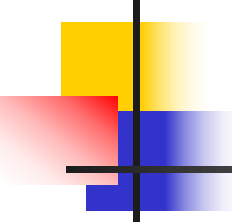
8.4.1 PageRank

PageRank简单计算：

假设一个只有由4个页面组成的集合：A、B、C和D。如果所有页面都链向A，那么A的PR（PageRank）值将是B、C及D的和。

$$PR(A)=PR(B)+PR(C)+PR(D)$$

继续假设B也有链接到C，并且D也有链接到A包括的3个页面。一个页面不能投票两次，所以B给每个页面半票。以同样的逻辑，D投出的票只有三分之一算到了A的PageRank上。



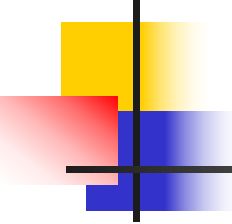
8.4.1 PageRank

$$PR(A) = \frac{PR(B)}{2} + \frac{PR(C)}{1} + \frac{PR(D)}{3}$$

换句话说，根据链出总数平分一个页面的**PR**值。

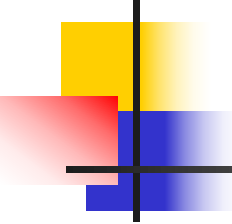
$$PR(A) = \frac{PR(B)}{L(B)} + \frac{PR(C)}{L(C)} + \frac{PR(D)}{L(D)}$$

如果应用在给社交网络上的用户推荐新人，这样就需要用到个性化**PageRank**算法进行个性化的定制。



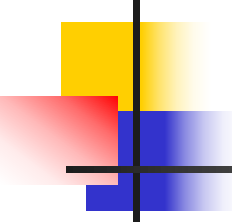
8.4.1 PageRank

个性化PageRank是PageRank的一个变种，目标是要计算所有节点相对于目标节点的相关度。从目标节点开始游走，每到一个节点都以 $1-d$ 的概率停止游走并从目标节点重新开始，或者以 d 的概率继续游走，从当前节点指向的节点中按照均匀分布随机选择一个节点往下游走。这样经过多轮游走之后，每个顶点被访问到的概率也会收敛趋于稳定，这时就可以用概率来进行排名了。当然个性化PageRank算法也有一些缺点：（1）只有一个源顶点可以被指定；（2）不能指定每个顶点的权值。



8.4.1 PageRank

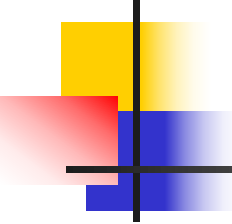
在社交网络数据集中可以使用PageRank算法设计计算每个用户的网页级别。例如，一组用户`/usr/local/spark-2.3.0-bin-hadoop27/data/graphx/users.txt`，以及用户之间的关系`/usr/local/spark-2.3.0-bin-hadoop2.7/data/graphx/followers.txt`。计算过程如代码8-17所示。



8.4.1 PageRank

代码8-17

```
package org.apache.spark.examples.graphx
import org.apache.spark.graphx.GraphLoader
import org.apache.spark.sql.SparkSession
object PageRankExample {
  def main(args: Array[String]): Unit = {
    // 创建一个 SparkSession.
    val spark = SparkSession
      .builder
      //如果代码在本地计算机运行需要添加 master("local")
      .appName(s"${this.getClass.getSimpleName}").master("local")
      .getOrCreate()
    val sc = spark.sparkContext
    // 加载边数据，创建 Graph
    val graph = GraphLoader.edgeListFile(sc, "followers.txt")
    // 运行 PageRank
    val ranks = graph.pageRank(0.0001).vertices
```



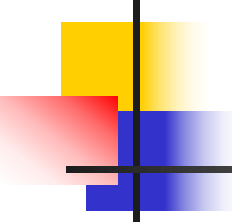
8.4.1 PageRank

续代码8-17

```
// 将排名与用户名连接，连接后输出结果
val users = sc.textFile("users.txt").map { line =>
    val fields = line.split(",")
    (fields(0).toLong, fields(1))
}
val ranksByUsername = users.join(ranks).map {
    case (id, (username, rank)) => (username, rank)
}
println(ranksByUsername.collect().mkString("\n"))
spark.stop()
}
```

运行该程序得到如下结果：

```
(justinbieber,0.15007622780470478)
(BarackObama,1.4596227918476916)
(matei_zaharia,0.7017164142469724)
(jeresig,0.9998520559494657)
(odersky,1.2979769092759237)
(ladygaga,1.3907556008752426)
```

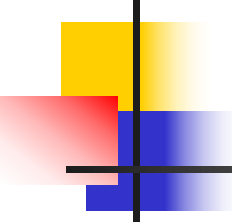


8.4.2 计算三角形数

通过计算三角形数可以衡量图或者子图的连通性，例如：

在一个社交网络中，如果每个人都影响其他人（每个人都连接到其他人），这样就会产生大量的三角形关系，也称为社区发现。

GraphX在对三角形计数时，会把图当作无向图。图或者子图有越多的三角形则连通性越好，这个性质可以确定小圈子，也可以用于提供推荐等。



8.4.2 计算三角形数

Triangle Count的算法思想如下：

- 计算每个节点的邻节点。
- 统计对每条边计算交集，并找出交集中id大于前两个节点id的节点。
- 对每个节点统计Triangle总数，注意只统计符合计算方向的Triangle Count。

接着通过一个社区网站用户之间关联情况的实例介绍Triangle Count的用法。followers.txt中为用户之间的关联情况，需要注意的是这些关联形成的图是有向的，字段之间用空格隔开。

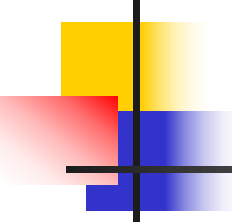


8.4.2 计算三角形数

用户followers.txt文档内容如图8-11所示。

```
2 1
4 1
1 2
6 3
7 3
7 6
6 7
3 7
```

图8-11 followers.txt文档

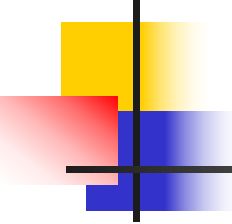


8.4.2 计算三角形数

users.txt中为社交网站的用户，该用户数据有三个字段，分别为序号，用户昵称，用户真实姓名，字段之间用逗号隔开。**users.txt**文档内容如图8-12所示。

```
1,BarackObama,Barack Obama  
2,ladygaga,Goddess of Love  
3,jeresig,John Resig  
4,justinbieber,Justin Bieber  
6,matei_zaharia,Matei Zaharia  
7,odersky,Martin Odersky  
8,anonsys
```

图8-12 users.txt文档



8.4.2 计算三角形数

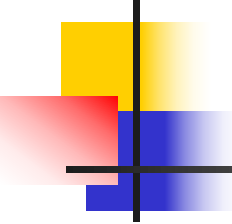
本实例的具体的程序以及注释如代码 8-18所示。

代码8-18

```
scala> import org.apache.spark.graphx.{GraphLoader, PartitionStrategy}
import org.apache.spark.graphx.{GraphLoader, PartitionStrategy}

scala> val graph = GraphLoader.edgeListFile(sc, "/usr/local/spark-2.3.0-bin-
hadoop2.7/data/graphx/followers.txt",
true).partitionBy(PartitionStrategy.RandomVertexCut)
graph: org.apache.spark.graphx.Graph[Int,Int] =
org.apache.spark.graphx.impl.GraphImpl@428169d

scala> val triCounts = graph.triangleCount().vertices //对每个顶点计算三角形数
triCounts: org.apache.spark.graphx.VertexRDD[Int] = VertexRDDImpl[128] at RDD at
VertexRDD.scala:57
```



8.4.2 计算三角形数

续代码**8-18**

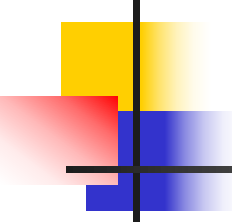
```
//将三角形数和用户名相联系
```

```
scala> val users = sc.textFile("/usr/local/spark-2.3.0-bin-  
hadoop2.7/data/graphx/users.txt").map {line =>  
    val fields = line.split(",")  
    (fields(0).toLong, fields(1))  
}
```

```
users: org.apache.spark.rdd.RDD[(Long, String)] = MapPartitionsRDD[133] at map at  
<console>:27
```

```
scala> val triCountByUsername = users.join(triCounts).map { case (id, (username, tc))  
=>  
    (username, tc)  
}
```

```
triCountByUsername: org.apache.spark.rdd.RDD[(String, Int)] =  
MapPartitionsRDD[137] at map at <console>:30
```

8.4.2 计算三角形数

续代码**8-18**

//输出结果

```
scala> println(triCountByUsername.collect().mkString("\n"))
```

```
(justinbieber,0)
```

```
(BarackObama,0)
```

```
(matei_zaharia,1)
```

```
(jeresig,1)
```

```
(odersky,1)
```

```
(ladygaga,0)
```

8.4.2 计算三角形数

结果可由followers.txt和users.txt文档内容所构建的图8-13看出，只有matei_zaharia、jeresig和odersky组成了三角形。

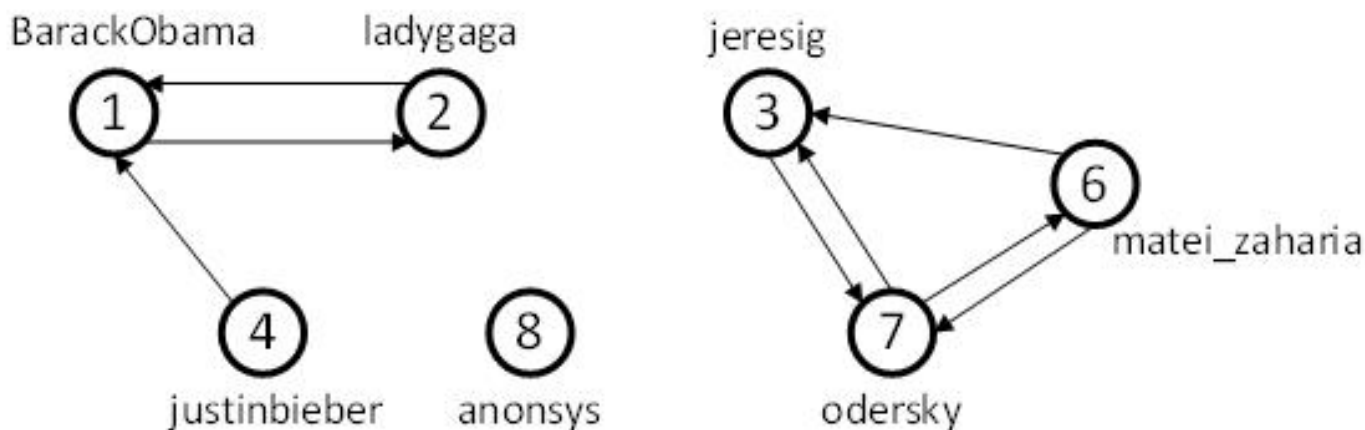
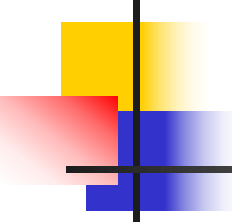


图8-13 三角形数示例图



8.4.3 计算连通分量

连通分量算法将图中的每个连通分量用其最小编号的顶点ID标记。例如，在社交网络中，连通分量类似集群，也就是朋友之间的小圈子。所以，使用连通分量算法能在社交网络中找到一些孤立的小圈子，并把他们在数据中心网络中区分开。在图8-14中找到连通分量，此图由7个顶点和5条边构成，其中有三个连通分量，每个连通分量可以认为是一个小圈子。

8.4.3 计算连通分量

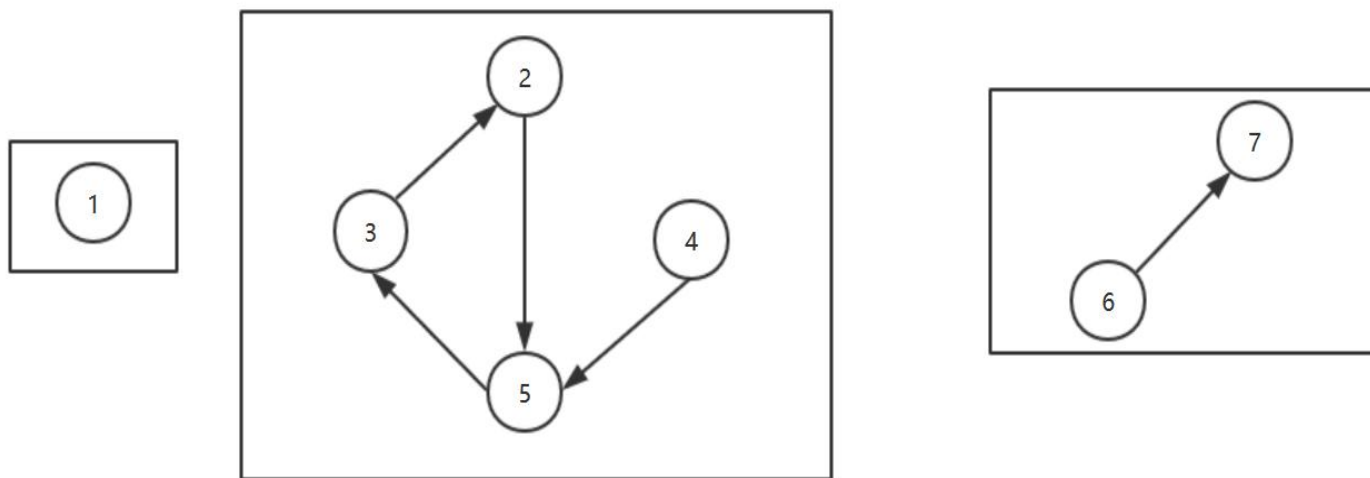
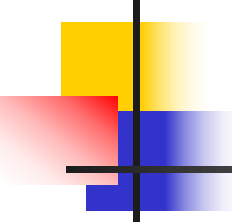


图8-14 连通分量示例图



8.4.3 计算连通分量

为了构建图8-15，首先用**Graph()**函数创建图，给每个顶点属性赋空值。然后调用**connectedComponents()**函数，其返回一个与输入的图对象结构相同的新**Graph**对象，连通分量使用其中最小的顶点**ID**标识，而这个最小的**ID**会赋值给这个连通分量中的每个顶点属性。例如，上图中的顶点**ID**为2，3，4，5的顶点组成一个连通分量，那它们的组件**ID**为其中的最小顶点**ID**，即**ID**为2，那么这个连通分量中的每个顶点属性都为2。

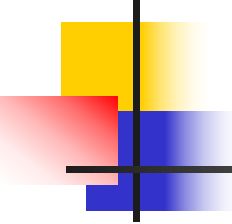


8.4.3 计算连通分量

具体实现如代码 8-19所示。

代码8-19

```
scala> import org.apache.spark.graphx._ //将GraphX包导入
import org.apache.spark.graphx._
//构建连通分量示例图，并缓存
scala> val g=Graph(sc.makeRDD((1L to 7L).map((_, ""))),
sc.makeRDD(Array(Edge(2L,5L, ""),Edge(5L,3L, ""),Edge(3L,2L, ""),
Edge(4L,5L, ""),Edge(6L,7L, "")))).cache
//使用连通分量算法，并通过map变换操作和ID分组显示结果
scala> g.connectedComponents.vertices.map(_.swap).groupByKey.map(_._2).collect
res12: Array[Iterable[org.apache.spark.graphx.VertexId]] = Array(CompactBuffer(1),
CompactBuffer(6, 7), CompactBuffer(4, 3, 5, 2))
```

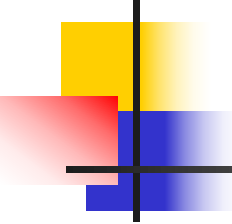


8.4.4 标签传播算法

标签传播算法（Label Propagation Algorithm, LPA）是一种基于图的半监督学习方法，主要用于社区发现。

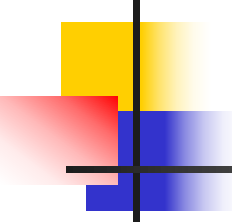
LPA算法思路简单清晰，其基本过程如下：

- 初始化。为每个顶点随机的指定一个自己特有的标签，在Spark文档中，指明用顶点的Id作为初始标签；



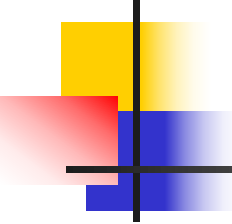
8.4.4 标签传播算法

- 更新所有顶点的标签。在每一次迭代中，每个顶点将自己的标签发送给它所有邻居，每个顶点根据收到的邻居消息进行标签更新，直到所有顶点的标签不再发生变化为止。对于每一轮迭代，节点标签的更新规则如下：对于某一个顶点，考察其所有邻居顶点的标签，并进行统计，将出现个数最多的那个标签赋值给当前顶点。当个数最多的标签不唯一时，随机选择一个标签赋值给当前顶点。



8.4.4 标签传播算法

由上述步骤可知，**LPA**算法中比较关键的部分为标签的更新过程，而且**LPA**并不关心边的方向，把图当成无向图。算法的每个迭代过程中顶点的标签更新是基于它的邻接顶点的标签。顶点 x 在选择标签时，如果存在多组个数最多的标签，则随机选择其中一组标签进行更新。



8.4.4 标签传播算法

然而，LPA常常不是收敛的，算法会在一直循环下去，对于这种情况，GraphX仅仅提供了一个静态方法，即指定迭代次数，并没有提供一个带公差终止条件的动态方法。

`org.apache.spark.graphx.lib.LabelPropagation`的`run()`方法，把图对象作为第一个参数，第二个参数为最大的迭代次数。

8.4.4 标签传播算法

将在图8-15中进行标签传播算法，这是个不收敛的例子，它的第五次迭代和第三次迭代结果相同。

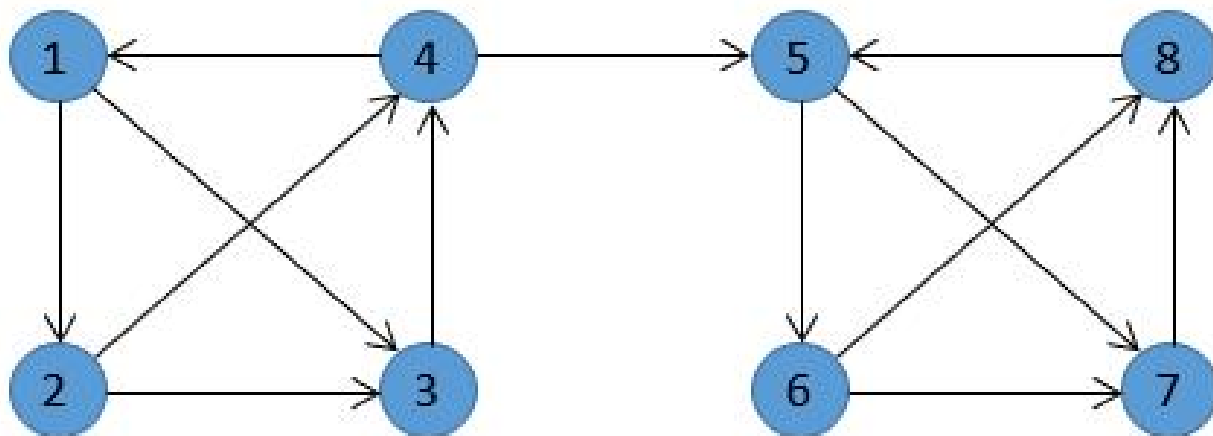
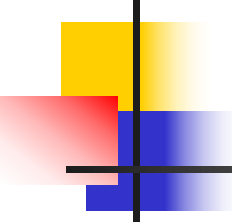


图8-15 LPA示例图



8.4.4 标签传播算法

具体实现如代码8-20所示。
代码**8-20**

```
scala> import org.apache.spark.graphx._
import org.apache.spark.graphx._

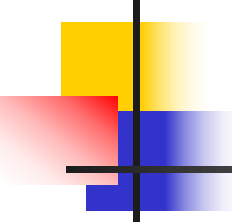
//构造VertexRDD
scala> val v =
sc.makeRDD(Array((1L,""),(2L,""),(3L,""),(4L,""),(5L,""),(6L,""),(7L,""),(8L,"")))
v: org.apache.spark.rdd.RDD[(Long, String)] = ParallelCollectionRDD[0] at makeRDD at
<console>:27
//构造EdgeRDD
scala> val e = sc.makeRDD(Array(Edge(1L,2L,""),Edge(2L,3L,""),Edge(3L,4L,""),
Edge(4L,1L,""),Edge(1L,3L,""),Edge(2L,4L,""),Edge(4L,5L,""),
Edge(5L,6L,""),Edge(6L,7L,""),Edge(7L,8L,""),Edge(8L,5L,""),
Edge(5L,7L,""),Edge(6L,8L,"")))
e: org.apache.spark.rdd.RDD[org.apache.spark.graphx.Edge[String]] =
ParallelCollectionRDD[1] at makeRDD at <console>:27
```



8.4.4 标签传播算法

续代码8-20

```
//调用LabelPropagation的run()方法，并根据顶点Id从小到大显示结果  
scala> lib.LabelPropagation.run(Graph(v,e),5).vertices.collect.sortWith(_. _1<_. _1)  
res13: Array[(org.apache.spark.graphx.VertexId, org.apache.spark.graphx.VertexId)] =  
Array((1,2), (2,1), (3,2), (4,1), (5,2), (6,1), (7,2), (8,3))
```



8.4.5 SVD++

本算法主要用于推荐系统，与机器学习相结合，属于监督性学习。

以设计一个推荐系统进行影片推荐为例，拥有用户对已经观看的影片历史评分，评分范围是1星到5星。如图8-16所示，左边顶点表示用户，右边顶点表示影片，边表示评分，预测用户4给电影3所打的分数。

8.4.5 SVD++

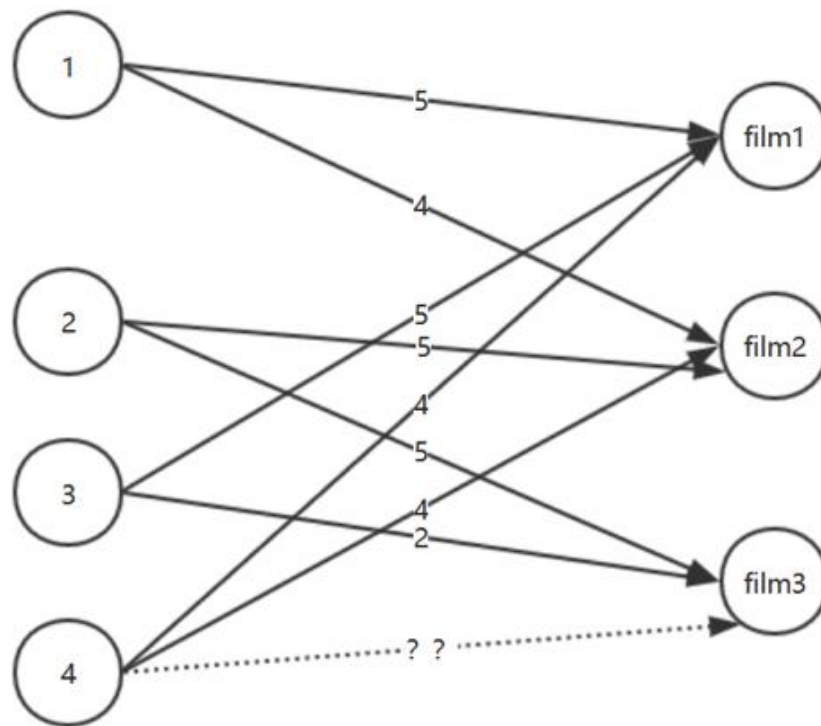
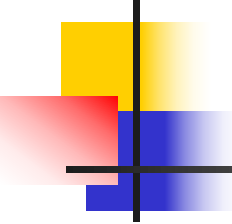
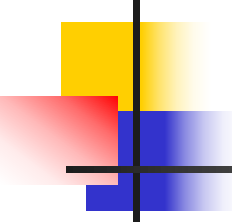


图8-16 影片推荐示例图



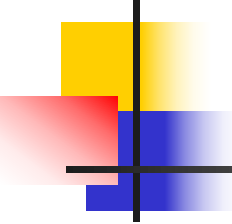
8.4.5 SVD++

解决这种问题一般的话有两种主流的方法。第一种主流方法比较直接：对于需要处理的用户3，找到和他有相同爱好的其他用户，然后给用户4推荐这些用户喜欢的影片，这种方法被称为邻居法，因为它使用了图中相邻用户的信息，这种方法也有缺点：有时比较难找到一个合适的邻居，同时这种方法也忽视了影片的一些潜在信息。



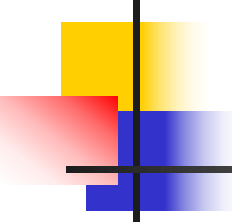
8.4.5 SVD++

第二种主流的方法就是取挖掘一些隐性变量，避免了第一种方法需要找到目标用户准确匹配的其他用户的要求。通过隐性变量，可以使用一个向量来表示每一部影片，向量表示电影拥有的不同特性，比如电影可以用一个二维向量来表示，第一个维度表示它属于科幻电影的程度，第二个维度表示它属于浪漫电影的程度。



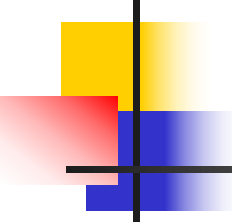
8.4.5 SVD++

第二种方法使用自动挖掘出来的隐性变量，考虑到了全局的信息，即使那些与目标用户不相似的用户，他们的喜好也会对每部电影隐性变量造成影响。这种方法的缺点就是在于不充分考虑用户自身的信息。**SVD++**算法就是基于隐性变量的基础上进行改进。



8.4.5 SVD++

具体实现如代码8-21所示，首先建立图形对象，然后调用**SVDPlusPlus**函数进行训练，**Conf**参数的设置及意义如表 8-11所示，训练之前参数就要确定下来，调参过程主要以经验为主，不断尝试。



8.4.5 SVD++

代码8-21

```
scala> import org.apache.spark.graphx._ //导入GraphX包
import org.apache.spark.graphx._

//构建图中的EdgeRDD
scala> val edges = sc.makeRDD (Array(Edge(1L,5L,5.0), Edge(1L,6L,4.0),
Edge(2L,6L,5.0), Edge(2L,7L,5.0), Edge(3L,5L,5.0), Edge(3L,6L,2.0), Edge(4L,5L,4.0),
Edge(4L,6L,4.0)))
edges: org.apache.spark.rdd.RDD[org.apache.spark.graphx.Edge[Double]] =
ParallelCollectionRDD[0] at makeRDD at <console>:27

//算法指定超参数，参考参数设定表
scala> val conf=new lib.SVDPlusPlus.Conf(2,10,0,5,0.007,0.007,0.005,0.015)
conf: org.apache.spark.graphx.lib.SVDPlusPlus.Conf =
org.apache.spark.graphx.lib.SVDPlusPlus$Conf@2674ca88
```

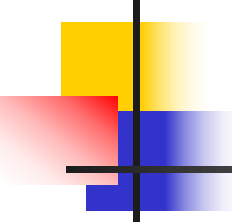


8.4.5 SVD++

续代码8-21

```
//运行SVD++算法，获得返回的模型—输入图的再处理结果和数据集的平均打分情况
scala> val (g,mean)=lib.SVDPlusPlus.run(edges,conf)
g: org.apache.spark.graphx.Graph[(Array[Double], Array[Double], Double, Double),Double]
= org.apache.spark.graphx.impl.GraphImpl@3ce4eb42
mean: Double = 4.25

//定义pred()函数，输入为模型的参数、用户id和需要预测的影片
scala> def pred (g:Graph[(Array[Double], Array[Double], Double, Double), Double ],
mean:Double, u:Long, i:Long)={
    val user=g.vertices.filter(_. _1 == u).collect()(0)._2
    val item=g.vertices.filter(_. _1 == i).collect()(0)._2
    mean+user._3+item._3+item._1.zip(user._2).map(x => x._1 * x._2).reduce(_+_ )
}
pred: (g: org.apache.spark.graphx.Graph[(Array[Double], Array[Double], Double,
Double),Double], mean: Double, u: Long, i: Long)Double
```



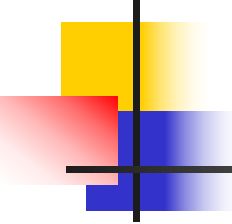
8.4.5 SVD++

续代码8-21

//SVDPlusPlus的一部分初始化是使用随机数的，所以对于同一份输入结果每次程序的预测结果不一定完全一样

```
scala> pred(g,mean,4L,7L)
```

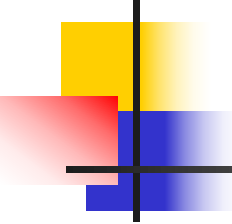
```
res14: Double = 5.935786807208753
```



8.4.5 SVDPlusPlus

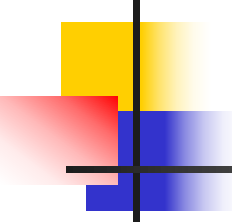
表8-11 Conf参数设定表

参数	示例	描述
Rank	2	隐性变量的个数
MaxIters	10	执行的迭代数,值越大越准确
minVal	0	最低评分
maxVal	5	最高评分
gamma1	0.007	每次迭代中偏差改变速度
gamma2	0.007	隐性变量的改变速度
gamma6	0.005	偏差的阻尼系数
gamma7	0.015	不同隐性变量相互影响程度



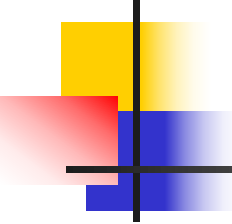
8.5 GraphX实现图经典算法

有一些与图有关的算法在GraphX还没有提供，本节就介绍几个经典的图算法在GraphX中的实现，包括Dijkstra算法、旅行推销员问题（TSP）和最小生成树算法。



8.5.1 Dijkstra算法

从图中的某个顶点出发到达另外一个顶点的所经过的边的权重和最小的一条路径，称为最短路径。**Dijkstra**算法使用了广度优先搜索解决赋权有向图或者无向图的单源最短路径问题（计算一个顶点到其他每个顶点的距离）。



8.5.1 Dijkstra算法

算法流程如下：

1. 初始时令 $S=\{V_0\}$, $T=V-S=\{\text{其余顶点}\}$, T 中顶点对应的距离值

若存在 $\langle V_0, V_i \rangle$, $d(V_0, V_i)$ 为 $\langle V_0, V_i \rangle$ 弧上的权值

若不存在 $\langle V_0, V_i \rangle$, $d(V_0, V_i)$ 为 ∞

2. 从 T 中选取一个与 S 中顶点有关联边且权值最小的顶点 W , 加入到 S 中

3. 对其余 T 中顶点的距离值进行修改：若加进 W 作中间顶点, 从 V_0 到 V_i 的距离值缩短, 则修改此距离值

重复上述步骤2、3, 直到 S 中包含所有顶点, 即 $W=V_i$ 为止

8.5.1 Dijkstra算法

对于图8-17，包含7个结点和11条路径，把节点A作为初始节点，在IDEA中运行Dijkstra算法，具体实现如代码8-22所示。

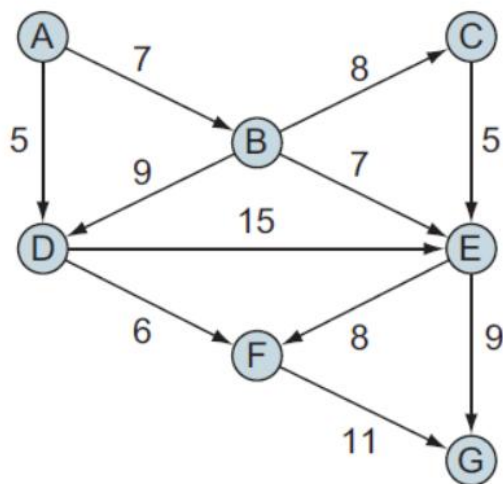
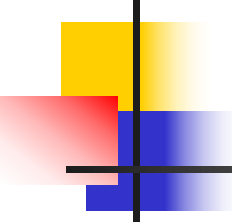


图8-17 经典算法示例图



8.5.1 Dijkstra算法

代码8-22

```
import org.apache.log4j.{Level, Logger}
import org.apache.spark.SparkConf
import org.apache.spark.SparkContext
import org.apache.spark.graphx.{Edge, Graph, TripletFields, VertexId}
import scala.reflect.ClassTag

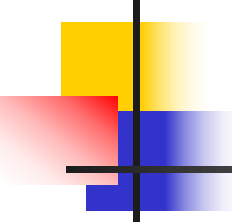
object Paths {      //单源最短路径
  def dijkstra[VD: ClassTag](g : Graph[VD, Double], origin: VertexId) = {
    //初始化，其中属性为（boolean, double, Long）类型，boolean 用于标记是否访问
    //过，double 为顶点距离原点的距离，Long 是上一个顶点的 id
    var g2 = g.mapVertices((vid, _) => (false, if(vid == origin) 0 else Double.MaxValue, -1L))
    for(i <- 1L to g.vertices.count()) {
      //从没有访问过的顶点中找出距离原点最近的点
      val currentVertexId = g2.vertices.filter(!_._2._1).reduce((a,b) => if (a._2._2 < b._2._2) a
      else b)._1
      //更新 currentVertexId 邻接顶点的 ‘double’ 值
      val newDistances = g2.aggregateMessages[(Double, Long)](
        triplet => if(triplet.srcId == currentVertexId && !triplet.dstAttr._1) {    //只给未确定
        的顶点发送消息
```



8.5.1 Dijkstra算法

续代码8-22

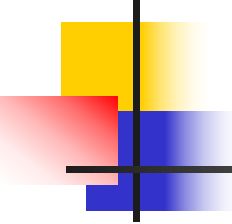
```
        triplet.sendToDst((triplet.srcAttr._2 + triplet.attr, triplet.srcId))
      },(x, y) => if(x._1 < y._1) x else y ,
      TripletFields.All)
    g2 = g2.outerJoinVertices(newDistances) {           //更新图形
      case (vid, vd, Some(newSum)) => (vd._1 ||
        vid == currentVertexId, math.min(vd._2, newSum._1), if(vd._2 <= newSum._1)
vd._3 else newSum._2 )
      case (vid, vd, None) => (vd._1 || vid == currentVertexId, vd._2, vd._3)
    }
  }
  g.outerJoinVertices(g2.vertices){ (vid, srcAttr, dist) => (srcAttr, dist.getOrElse(false,
Double.MaxValue, -1)._2) }
}
def main(args: Array[String]): Unit ={
  val conf = new SparkConf().setAppName("ShortPaths").setMaster("local[4]")//指定四个
本地线程数目，来模拟分布式集群
  val sc = new SparkContext(conf) //屏蔽日志
```



8.5.1 Dijkstra算法

续代码8-22

```
Logger.getLogger("org.apache.spark").setLevel(Level.WARN)
Logger.getLogger("org.eclipse.jetty.server").setLevel(Level.OFF)
val myVertices = sc.makeRDD(Array((1L, "A"), (2L, "B"), (3L, "C"), (4L, "D"), (5L, "E"), (6L,
"F"), (7L, "G")))
val initialEdges = sc.makeRDD(Array(Edge(1L, 2L, 7.0), Edge(1L, 4L, 5.0),
    Edge(2L, 3L, 8.0), Edge(2L, 4L, 9.0), Edge(2L, 5L, 7.0), Edge(3L, 5L, 5.0), Edge(4L, 5L, 15.0),
Edge(4L, 6L, 6.0), Edge(5L, 6L, 8.0), Edge(5L, 7L, 9.0), Edge(6L, 7L, 11.0)))
val myEdges = initialEdges.filter(e => e.srcId != e.dstId).flatMap(e => Array(e, Edge(e.dstId,
e.srcId, e.attr))).distinct() //去掉自循环边，有向图变为无向图，去除重复边
val myGraph = Graph(myVertices, myEdges).cache()
println(dijkstra(myGraph, 1L).vertices.map(x => (x._1, x._2)).collect().mkString(" | "))
}
}
```

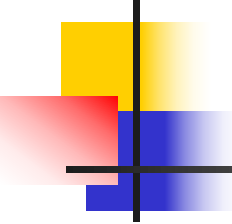


8.5.1 Dijkstra算法

代码解释：

Dijkstra最短路径算法的实现，用了**var**变量定义**g2**，这是因为迭代算法要把每一轮的迭代计算结果赋值给一个变量以便在下一轮迭代中使用。

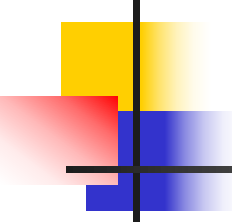
首先初始化变量**g2**，去掉原来的图**g**的顶点数据，添加由**Boolean**和**Double**组成的键值对，**Boolean**变量标识顶点是否被访问过，**Double**表示源顶点到当前顶点的距离。



8.5.1 Dijkstra算法

当计算出距离值后，调用 `outerJoinVertices()` 函数更新距离值生成一个新图，并赋值给 `g2`。然后 `Dijkstra` 函数中最后一行就是把新图 `g2` 的顶点属性重新保存到原来的图 `g` 中。

最后在主函数中定义图的顶点和边，然后选择初始节点，调用函数打印出输出结果即可。

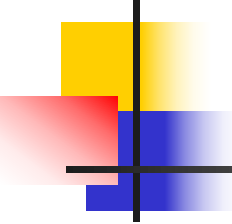


8.5.1 Dijkstra算法

结果输出为:

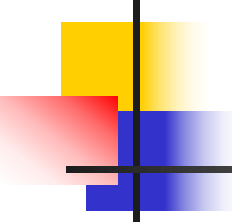
$(4,(D,5.0)) \mid (1,(A,0.0)) \mid (5,(E,14.0)) \mid (6,(F,11.0)) \mid$
 $(2,(B,7.0)) \mid (3,(C,15.0)) \mid (7,(G,22.0))$

结果说明: 以 $(4,(D,5.0))$ 为例, 表示ID为4的顶点D到顶点A的最短距离为5.0。



8.5.2 TSP问题

TSP问题（Traveling Salesman Problem，旅行商问题），由威廉哈密顿爵士和英国数学家克克曼 T.P.Kirkman 于 19 世纪初提出。问题描述如下：有若干个城市，任何两个城市之间的距离都是确定的，现要求一旅行商从某城市出发必须经过每一个城市且只在一个城市逗留一次，最后回到出发的城市，问如何事先确定一条最短的线路已保证其旅行的费用最少？

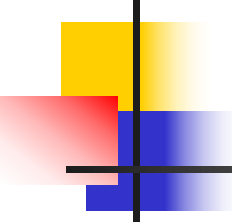


8.5.2 TSP问题

TSP问题是一个在无向图中找到一个经过每一个顶点的最短路径，至今没有一个简单的确定算法来解决这种NP-hard问题。最简单的解决方法就是用贪心算法求解，但是由于贪心算法会产生偏离最优解的答案，并且不一定会到达所有的顶点。

算法流程如下：

- (1) 从某些节点开始；
- (2) 添加权重最小的临边到生成树；
- (3) 跳转到第2步。



8.5.2 TSP问题

根据图8-17的TSP算法具体实现如代码8-23所示。

代码8-23

```
import org.apache.log4j.{Level, Logger}
import org.apache.spark.{SparkConf, SparkContext}
import org.apache.spark.graphx._
object TSP {
  def greedy[VD](g: Graph[VD, Double], origin: VertexId) = {
    var g2: Graph[Boolean, (Double, Boolean)] = g.mapVertices((vid, vd) => vid ==
origin).mapTriplets {
      et => (et.attr, false)
    }
    var nextVertexId = origin
    var edgesAreAvailable = true
    type tripletType = EdgeTriplet[Boolean, (Double, Boolean)]
    do {
      val availableEdges = g2.triplets.filter { et => !et.attr._2 && (et.srcId == nextVertexId
&& !et.dstAttr || et.dstId == nextVertexId && !et.srcAttr) }
```

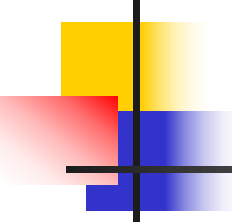


8.5.2 TSP问题

续代码**8-23**

```
edgesAreAvailable = availableEdges.count > 0
if (edgesAreAvailable) {
  val smallestEdge = availableEdges.min()(new Ordering[tripletType]() {
    override def compare(a: tripletType, b: tripletType) = {
      Ordering[Double].compare(a.attr._1, b.attr._1)
    }
  })
  nextVertexId = Seq(smallestEdge.srcId, smallestEdge.dstId).filter(_ !=
nextVertexId).head
  g2 = g2.mapVertices((vid, vd) => vd || vid == nextVertexId).mapTriplets { et =>
    (et.attr._1, et.attr._2 ||
      (et.srcId == smallestEdge.srcId
        && et.dstId == smallestEdge.dstId))
  }
} while (edgesAreAvailable)
g2
}

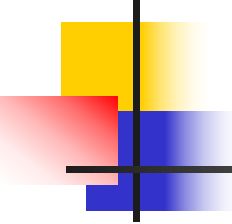
def main(args: Array[String]): Unit = {
  val conf = new SparkConf().setAppName("ShortPaths").setMaster("local[4]")
  val sc = new SparkContext(conf) //屏蔽日志
```



8.5.2 TSP问题

续代码8-23

```
Logger.getLogger("org.apache.spark").setLevel(Level.WARN)
Logger.getLogger("org.eclipse.jetty.server").setLevel(Level.OFF)
val myVertices = sc.makeRDD(Array((1L, "A"), (2L, "B"), (3L, "C"), (4L, "D"), (5L, "E"), (6L,
"F"), (7L, "G")))
val initialEdges = sc.makeRDD(Array(Edge(1L, 2L, 7.0), Edge(1L, 4L, 5.0),
    Edge(2L, 3L, 8.0), Edge(2L, 4L, 9.0), Edge(2L, 5L, 7.0), Edge(3L, 5L, 5.0), Edge(4L, 5L, 15.0),
Edge(4L, 6L, 6.0),
    Edge(5L, 6L, 8.0), Edge(5L, 7L, 9.0), Edge(6L, 7L, 11.0)))
val myEdges = initialEdges.filter(e => e.srcId != e.dstId).flatMap(e => Array(e, Edge(e.dstId,
e.srcId, e.attr))).distinct()
val myGraph = Graph(myVertices, myEdges).cache()
println(greedy(myGraph, 1L).vertices.map(x => (x._1, x._2)).collect().mkString(" | "))
}
}
```

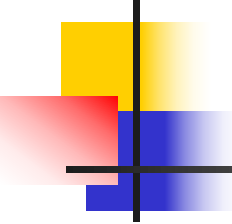


8.5.2 TSP问题

程序运行结果为：

(4,true) | (1,true) | (5,true) | (6,true) | (2,true) | (3,true) |
(7,false)

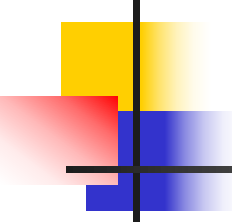
结果说明：可以成功到达顶点ID为4,1,5,6,2,3的顶点，顶点ID为7的顶点没有到达。



8.5.3 最小生成树问题

最小生成树的定义为在连通网的所有生成树中，所有边的代价和最小的生成树，称为最小生成树。解决这个问题主要方法由Kruskal和Prime，这里主要用Prime求解。

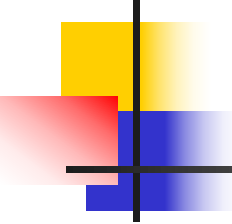
Prime算法也可以称为“加点法”，每次迭代选择代价最小的边对应的点，加入到最小生成树中。算法从某一个顶点 s 开始，逐渐长大覆盖整个连通网的所有顶点。



8.5.3 最小生成树问题

设 $G=(V,E)$ 是连通带权图, $V=\{1,2,\dots,n\}$, 构造 G 的最小生成树的Prim算法的基本思想是:

- (1) 置 $S=\{1\}$;
- (2) 只要 S 是 V 的真子集, 就作如下的贪心选择: 选取满足条件 $i \in S, j \in V-S$, 且 $c[i][j]$ 最小的边, 将顶点 j 添加到 S 中, 直到 $S=V$ 时为止;
- (3) 选取到的所有边恰好构成 G 的一棵最小生成树。



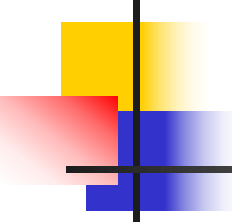
8.5.3 最小生成树问题

根据图8-17的Prime算法具体实现如代码8-24所示。

代码8-24

```
import TSP.greedy
import org.apache.log4j.{Level, Logger}
import org.apache.spark.{SparkConf, SparkContext}
import org.apache.spark.graphx._

object Prime {
  //最小生成树
  def prime[VD: scala.reflect.ClassTag](g: Graph[VD, Double], origin: VertexId) = {
    //初始化,其中属性为 (boolean, double, Long) 类型, boolean 用于标记是否访问过,
    //double 为加入当前顶点的代价, Long 是上一个顶点的 id
    var g2 = g.mapVertices((vid, _) => (false, if (vid == origin) 0 else Double.MaxValue, -1L))
    for (i <- 1L to g.vertices.count()) {
      //从没有访问过的顶点中找出 代价最小的点
      val currentVertexId = g2.vertices.filter(!_._2._1).reduce((a, b) => if (a._2._2 < b._2._2) a
        else b)._1
      //更新 currentVertexId 邻接顶点的 “double” 值
      val newDistances = g2.aggregateMessages[(Double, Long)](
        triplet => if (triplet.srcId == currentVertexId && !triplet.dstAttr._1) { //只给未确定的
          顶点发送消息
        }
      )
    }
  }
}
```

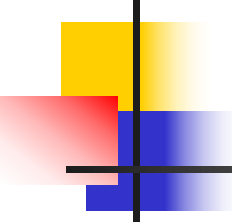


8.5.3 最小生成树问题

续代码8-24

```
        triplet.sendToDst((triplet.attr, triplet.srcId))
    },
    (x, y) => if (x._1 < y._1) x else y,
    TripletFields.All
)
//更新图形
g2 = g2.outerJoinVertices(newDistances) {
    case (vid, vd, Some(newSum)) => (vd._1 || vid == currentVertexId, math.min(vd._2,
        newSum._1), if (vd._2 <= newSum._1) vd._3 else newSum._2)
    case (vid, vd, None) => (vd._1 || vid == currentVertexId, vd._2, vd._3)
}
}
g2.outerJoinVertices(g2.vertices)((vid, srcAttr, dist) => (srcAttr, dist.getOrElse(false,
    Double.MaxValue, -1)._2,
    dist.getOrElse(false, Double.MaxValue, -1)._3))
}

def main(args: Array[String]): Unit = {
    val conf = new SparkConf().setAppName("ShortPaths").setMaster("local[4]")
    val sc = new SparkContext(conf) //屏蔽日志
    Logger.getLogger("org.apache.spark").setLevel(Level.WARN)
    Logger.getLogger("org.eclipse.jetty.server").setLevel(Level.OFF)
```



8.5.3 最小生成树问题

续代码**8-24**

```
val myVertices = sc.makeRDD(Array((1L, "A"), (2L, "B"), (3L, "C"), (4L, "D"), (5L, "E"), (6L,
    "F"), (7L, "G")))
val initialEdges = sc.makeRDD(Array(Edge(1L, 2L, 7.0), Edge(1L, 4L, 5.0),
    Edge(2L, 3L, 8.0), Edge(2L, 4L, 9.0), Edge(2L, 5L, 7.0), Edge(3L, 5L, 5.0),
    Edge(4L, 5L, 15.0), Edge(4L, 6L, 6.0),
    Edge(5L, 6L, 8.0), Edge(5L, 7L, 9.0), Edge(6L, 7L, 11.0)))
val myEdges = initialEdges.filter(e => e.srcId != e.dstId).flatMap(e => Array(e,
    Edge(e.dstId, e.srcId, e.attr))).distinct()
val myGraph = Graph(myVertices, myEdges).cache()
println(prime(myGraph, 1L).vertices.map(x => (x._1, x._2)).collect().mkString(" | "))
}
}
```

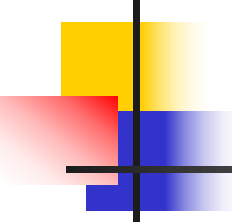


8.5.3 最小生成树问题

运行结果：

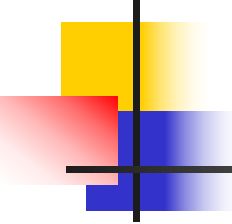
(4,(D,5.0,1)) | (1,(A,0.0,-1)) | (5,(E,7.0,2)) |
(6,(F,6.0,4)) | (2,(B,7.0,1)) | (3,(C,5.0,5)) | (7,(G,9.0,5))

结果说明：结果显示的是依次添加的边集无先后顺序且无方向，如第一项(4,(D,5.0,1))表示为顶点ID为4的顶点和顶点ID为1 的顶点相连，它们边的属性为5.0，第二项(1,(A,0.0,-1))中-1表示A不与其他节点相连。



8.6 GraphX实例分析

本节通过两个实例来展示GraphX的具体应用场景。包括寻找“最有影响力”论文和寻找社交媒体中的“影响力用户”。

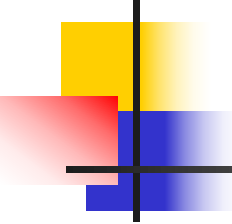


8.6.1 寻找“最有影响力”论文

首先准备要用到的数据，本实例选用高能物理理论应用网络数据集，具体下载界面地址为：

<http://snap.stanford.edu/data/cit-HepTh.html>。下载完成并解压后即可放到相应的目录。

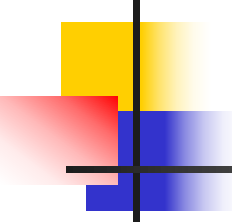
本书中将Cit-HepTh.txt文件放在Idea对应的project中，Cit-HepTh.txt文件格式如图8-18所示。



8.6.1 寻找“最有影响力”论文

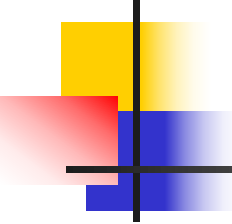
```
# Directed graph (each unordered pair of nodes is saved once): Cit-HepTh.txt
# Paper citation network of Arxiv High Energy Physics Theory category
# Nodes: 27770 Edges: 352807
# FromNodeId ToNodeId
1001    9304045
1001    9308122
1001    9309097
1001    9311042
1001    9401139
1001    9404151
.....
```

图8-18 Cit-HepTh.txt文件格式



8.6.1 寻找“最有影响力”论文

每一行数据表示图中的一条边，每一条边有源顶点ID和目标顶点ID，每个ID对应一篇论文。源顶点表示比较新的论文，目标顶点表示被引用的旧的论文，论文的引用关系用边来表示，即新论文引用旧论文。如代码8-25所示。

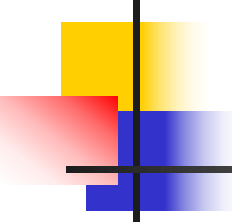


8.6.1 寻找“最有影响力”论文

代码8-25

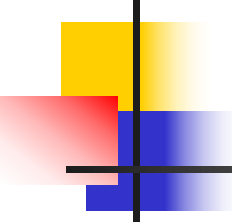
```
scala> import org.apache.spark.graphx._ //导入GraphX包
//文件路径根据存放位置而定
scala> val graph=GraphLoader.edgeListFile(sc, "/home/ubuntu/Cit-HepTh.txt")
scala> graph.inDegrees.reduce((a,b)=>if(a._2>b._2) a else b)
res15: (org.apache.spark.graphx.VertexId, Int) = (9711200,2414)
```

GraphLoader是GraphX类库里面的对象，通过import将其导入，然后通过函数edgeListFile加载边列表格式的文件。edgeListFile有两个参数，第一个是SparkContext参数sc，sc由spark-shell在初始化的时候创建，第二个参数是边列表文件的路径名称。



8.6.1 寻找“最有影响力”论文

变量`graph`通过调用`inDegrees`函数可得到(顶点ID,入度)的`VertexRDD`，此`RDD`提供一个`reduce()`函数，此函数只有一个参数，该参数本身就是一个匿名函数，引用此函数就可以把`RDD`的两条数据归并成一个结果，归并的原则就是选取入度大的顶点。结论所示：**ID为9711200的论文被其他论文引用2414次，引用次数最多。**



8.6.1 寻找“最有影响力”论文

接下来，运用`graph`变量的`pageRank()`函数返回一个新的`Graph`，其中每个点有`Double`类型的属性值（原始的图`edgeListFile`给顶点属性赋予一个默认值1），参数0.001是为了平衡速度和最终结果准确度之间的一个容忍度数值。然后在结果集`v`上运行`reduce()`归并函数，找出PR值最高的顶点。具体实现如代码8-26所示。

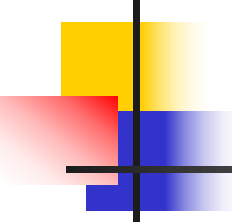


8.6.1 寻找“最有影响力”论文

代码8-26

```
scala> graph.vertices.take(5) //首先查看图中的顶点数据格式，顶点属性默认值为1
res16: Array[(org.apache.spark.graphx.VertexId, Int)] = Array((9405166,1), (108150,1), (110163,1), (204100,1), (9407099,1))
scala> val v=graph.pageRank(0.001).vertices
scala> v.reduce((a,b)=>if(a._2>b._2) a else b)
res17: (org.apache.spark.graphx.VertexId, Double) = (9207016,85.27317386053808)
```

结果显示编号为9207016论文的PR值最高，是最具有影响力的论文。



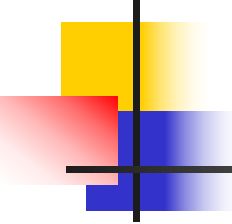
8.6.1 寻找“最有影响力”论文

接着实现个性化的PageRank。以编号为9207016论文为例，要找到对其影响最大的论文，具体实现如代码8-27所示。

代码8-27

```
scala>  
graph.personalizedPageRank(9207016,0.001).vertices.filter(_._1!=9207016).reduce((a,b) => if(a._2>b._2) a else b)  
res20: (org.apache.spark.graphx.VertexId, Double) =  
(9201015,0.09211875000000003)
```

结果显示编号为9201015的论文是对编号为9207016论文最重要的论文。

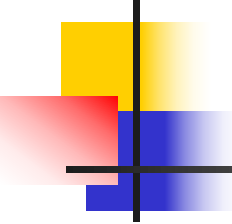


8.6.2 寻找社交媒体中的“影响力用户”

本实例是在求Twitter数据中，寻找“影响力用户”，简单而言就是找图中出度最大的节点。一个独立的推特用户可以通过他/她的推文影响到N个级别，即**followers of followers of followers...**。本个实例只考虑2级，即一个用户的**followers of followers**。

数据文件**twitter-graph-data.txt**下载链接地址为：
<https://github.com/knoldus/spark-graphx-twitter/tree/master/src/main/resources>。

在**txt**文件中每一行代表一个关系，前面一个是**followee**名称和id，后一个是**follower**的名称和id，其中用逗号隔开。图中的箭头是从**followee**指向**follower**，所以也可以表示成寻找被关注最多的人。



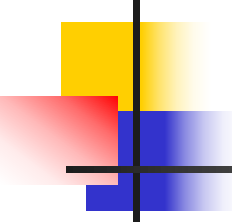
8.6.2 寻找社交媒体中的“影响力用户”

twitter-graph-data.txt文件内容如图8-19所示。

```
((User47,86566510),(User83,15647839))  
((User47,86566510),(User42,197134784))  
((User89,74286565),(User49,19315174))  
((User16,22679419),(User69,45705189))  
((User37,14559570),(User64,24742040))  
((User31,63644892),(User10,123004655))  
((User10,123004655),(User50,17613979))  
.....
```

图8-19 twitter-graph-data.txt文件格式

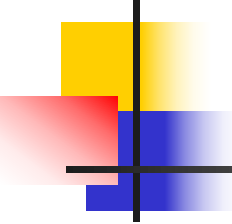
由于图中没有给每一条边的属性，所以默认就是**1**，或者也可以赋值成其他的一些提示信息。具体实现如代码8-28所示。



8.6.2 寻找社交媒体中的“影响力用户”

代码8-28

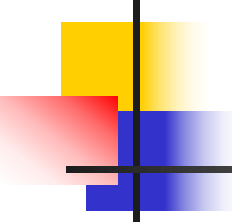
```
import java.io.PrintWriter
import org.apache.spark.graphx.{Edge, EdgeDirection, Graph, VertexId}
import org.apache.spark.rdd.RDD
import org.apache.spark.{SparkConf, SparkContext}
object Twitter_test {
  def main(args: Array[String]): Unit = {
    val conf = new SparkConf().setAppName("Twittter Influencer").setMaster("local[*]")
    val sparkContext = new SparkContext(conf)
    sparkContext.setLogLevel("ERROR")
    //文本文件的路径根实际存放的位置决定
    val twitterData = sparkContext.textFile("twitter-graph-data.txt")
    //分别从文本文件中提取 followee 和 follower 的数据
    val followeeVertices: RDD[(VertexId, String)] = twitterData.map(_._split(",")).map { arr =>
      val user = arr(0).replace("(" , "")
      val id = arr(1).replace(")", "")
      (id.toLong, user)
    }
  }
}
```



8.6.2 寻找社交媒体中的“影响力用户”

续代码8-28

```
val followerVertices: RDD[(VertexId, String)] = twitterData.map(_.split(",")).map { arr =>
    val user = arr(2).replace("(", "")
    val id = arr(3).replace(")", "")
    (id.toLong, user)
}
//接下来，使用 Spark GraphX API 从提取的数据创建图形。
val vertices = followeeVertices.union(followerVertices)
val edges: RDD[Edge[String]] = twitterData.map(_.split(",")).map { arr =>
    val followeeId = arr(1).replace(")", "").toLong
    val followerId = arr(3).replace(")", "").toLong
    Edge(followeeId, followerId, "follow")
}
val defaultUser = ("") //提供了一个默认输入
val graph = Graph(vertices, edges, defaultUser)
//使用 Spark GraphX 的 Pregel API 和广度优先遍历算法
val subGraph = graph.pregel("", 2, EdgeDirection.In)((_, attr, msg) =>
    attr + "," + msg,
    triplet => Iterator((triplet.srcId, triplet.dstAttr)),
    (a, b) => (a + "," + b))
```



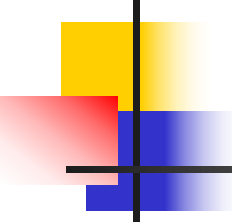
8.6.2 寻找社交媒体中的“影响力用户”

续代码8-28

```
//找到拥有最多 followers of followers 的用户
val lengthRDD = subGraph.vertices.map(vertex => (vertex._1,
    vertex._2.split(",").distinct.length - 2))
    .max()(new Ordering[Tuple2[VertexId, Int]]() {
        override def compare(x: (VertexId, Int), y: (VertexId, Int)): Int =
            Ordering[Int].compare(x._2, y._2)
    })

val userId = graph.vertices.filter(_._1 == lengthRDD._1).map(_._2).collect().head
println(userId + " has maximum influence on network with " + lengthRDD._2 + "
    influencers.")

val pw=new PrintWriter("Twitter_graph.gexf");
pw.close()
sparkContext.stop()
}
}
```



8.6.2 寻找社交媒体中的“影响力用户”

代码解释：首先就是导入数据并进行数据规格化，把txt文件中的逗号和括号都去除，把点和边都规格化了以后，再用`apply()`函数来构造一个图，利用`pregel`来形成一个子图，

输出结果为：

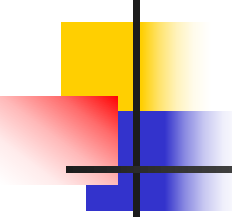
User36 has maximum influence on network with 95 influencers.

同样地，如果想要找到最具影响力的用户到N级，那么只需将Pregel API中的迭代次数从2增加到N。



8.7 本章小结

GraphX凭借Spark整体的一体化流水线处理、社区热烈的活跃度及快速的改进速度，具有强大的竞争力。GraphX框架本身是依托Spark平台构建了一个良好的图计算框架，若要用图计算解决实际问题，还需要将项目中实际遇到的诸多问题涉及的处理算法进行并行优化。本章对Spark GraphX中提供的一些图算法和实现进行了详细介绍，为图分布式计算提供了基础。



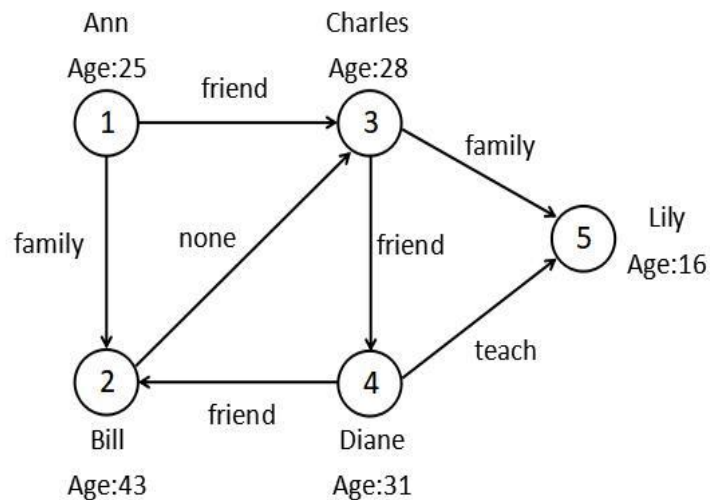
思考与习题

- 1、GraphX的基本数据结构包含了哪些类型的RDD?
- 2、简述GraphX的存储方式。
- 3、简述GraphX的实现架构。
- 4、aggregateMessages()方法和Pregel API之间有什么相同点和不同点?
- 5、简述PageRank算法的计算流程。
- 6、标签传播算法是如何实现的?
- 7、扩展本章介绍的Dijkstra算法，实现包含路径记录的Dijkstra最短路径算法。
- 8、本章实现了最小生成树的Prime算法，请实现另一个最小生成树Kruskal算法。

思考与习题

9. 下图中5个节点表示五个人，每个人有名字和年龄，边代表这些人之间的关系，每条边有其属性，

并根据此图完成下列要求：



(1) 构建关系图graph;

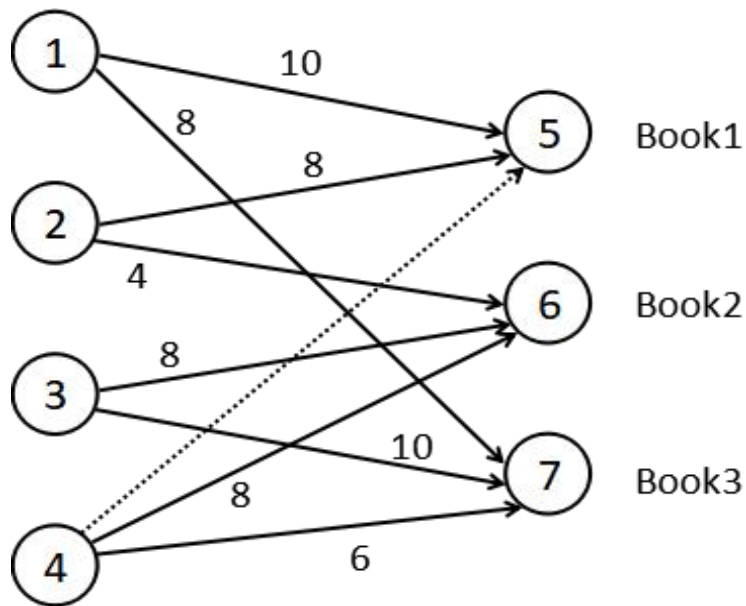
(2) 找出图中年龄大于20并小于30的顶点，并以"name is age"形式输出结果,例如: Ann is 25;

(3) 将图中所有顶点年龄加15，并以"Id is (name,age)"输出，例如: 1 is (Ann,40);

(4) 构建顶点年龄大于30的子图并输出子图所有的顶点和子图所有的边，输出顶点格式为"Ann is 25"形式，输出边格式为"1 to 3 attr friend".

思考与习题

10、设计一个书籍推荐系统，下图是一些用户对已阅读书籍的历史评分，评分范围是0-10分，现预测用户4会给Book1打多少分？





Spark大数据 编程基础 (Scala版)

